

Arquitectura Horizontal

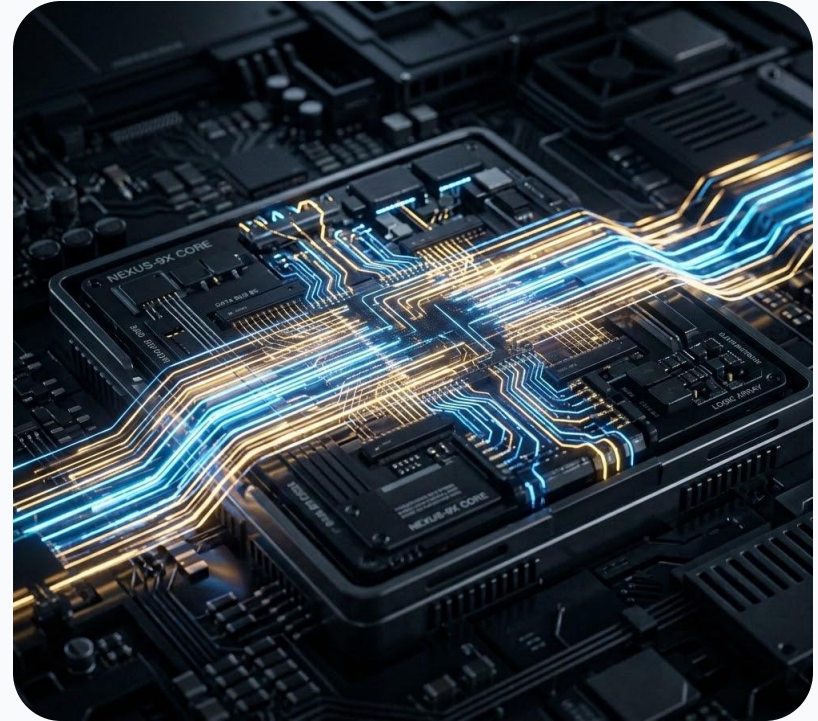
Ejecución del microprograma



Fundamentos de Arquitectura de computadoras

El control del procesador. Análisis de la secuencias de pasos del micro programa para la ejecución de un programa de usuario.

- Control Store
- Micro instrucción
- Microprograma



Máquina multinivel

NIVEL 5

Nivel de lenguaje orientado al problema

Traducción (Compilador)

NIVEL 4

Nivel de lenguaje ensamblador

Traducción (Ensamblador)

NIVEL 3

Nivel de máquina del sistema operativo

Interpretación parcial (S.O.)

NIVEL 2

Nivel de máquina convencional

Interpretación o Ejecución Directa

NIVEL 1

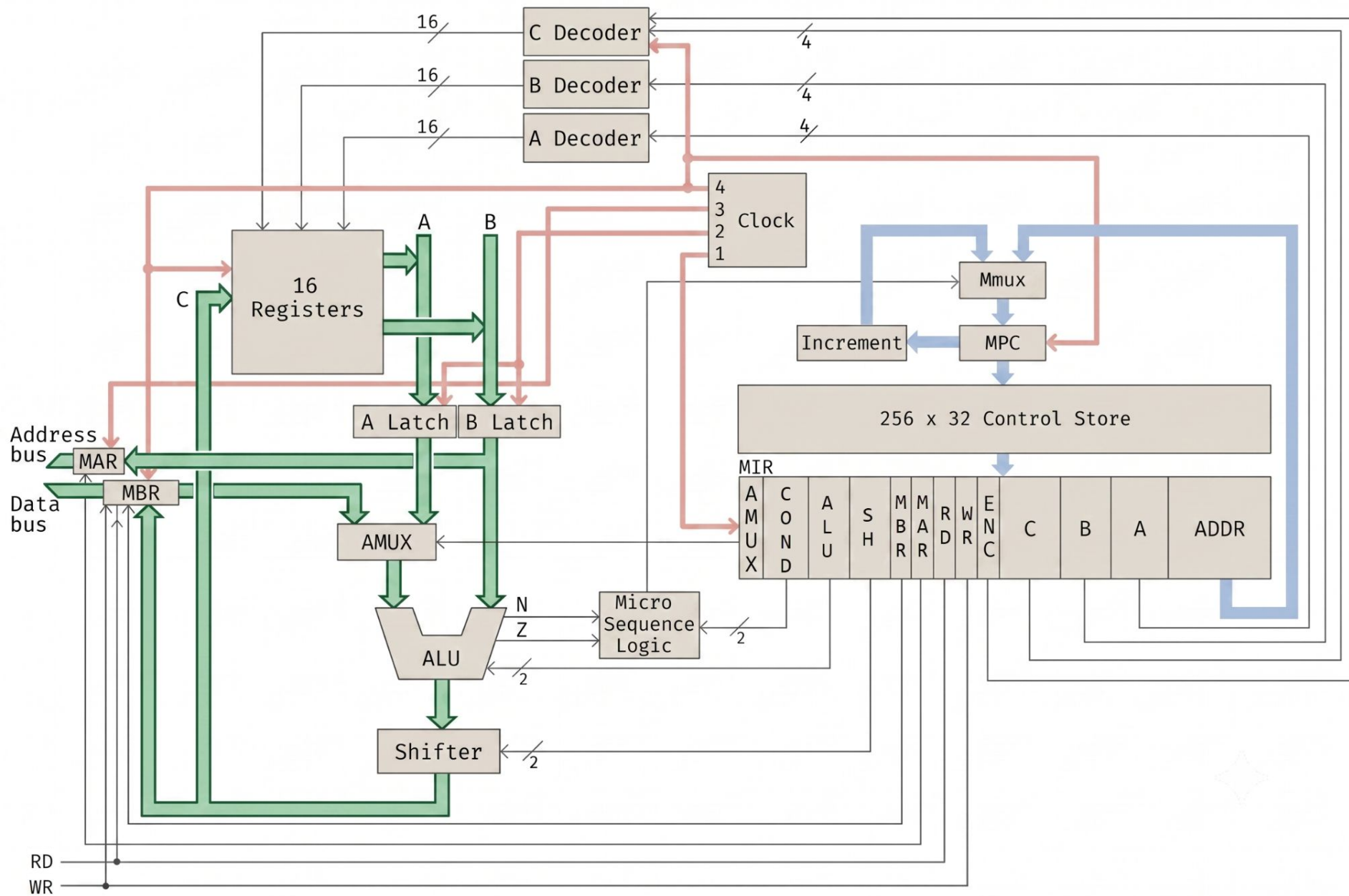
Nivel de microarquitectura

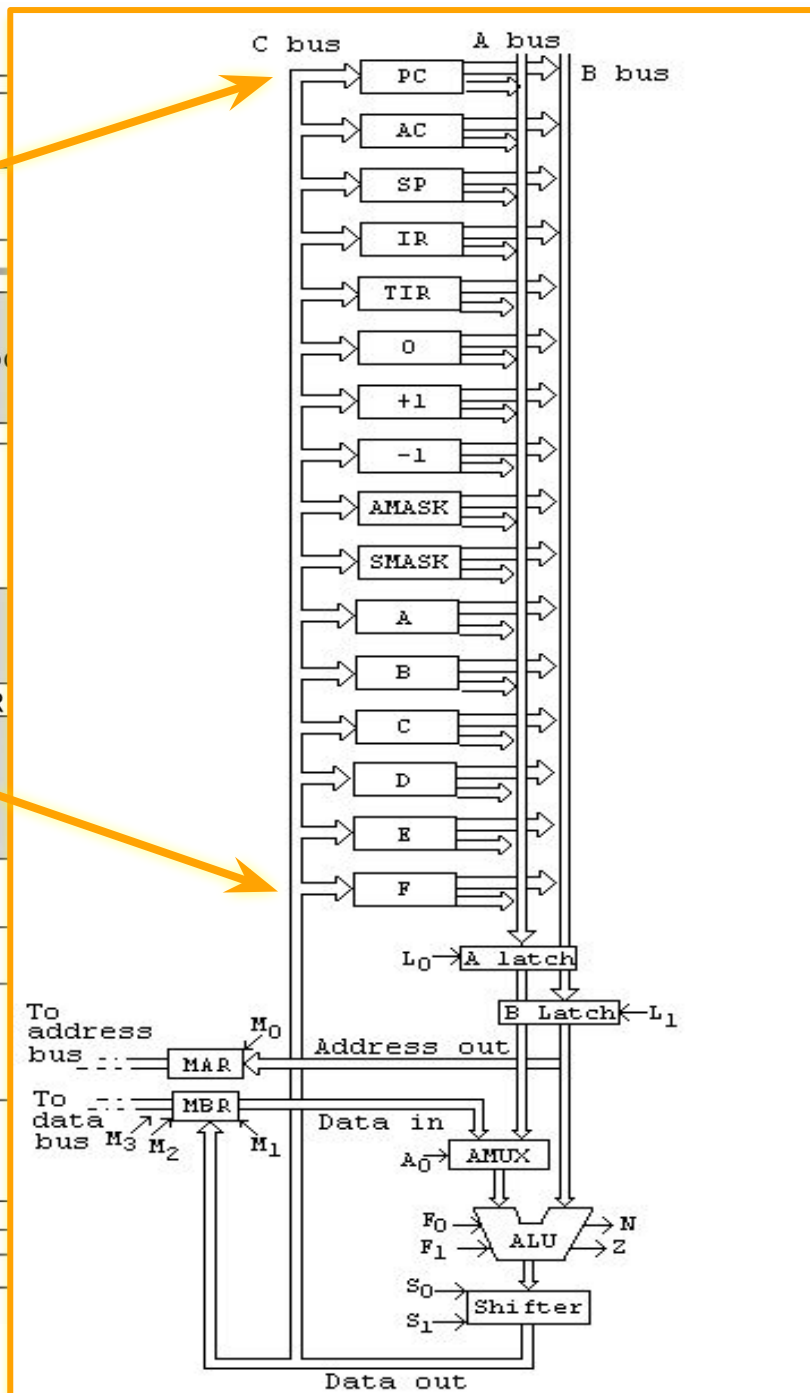
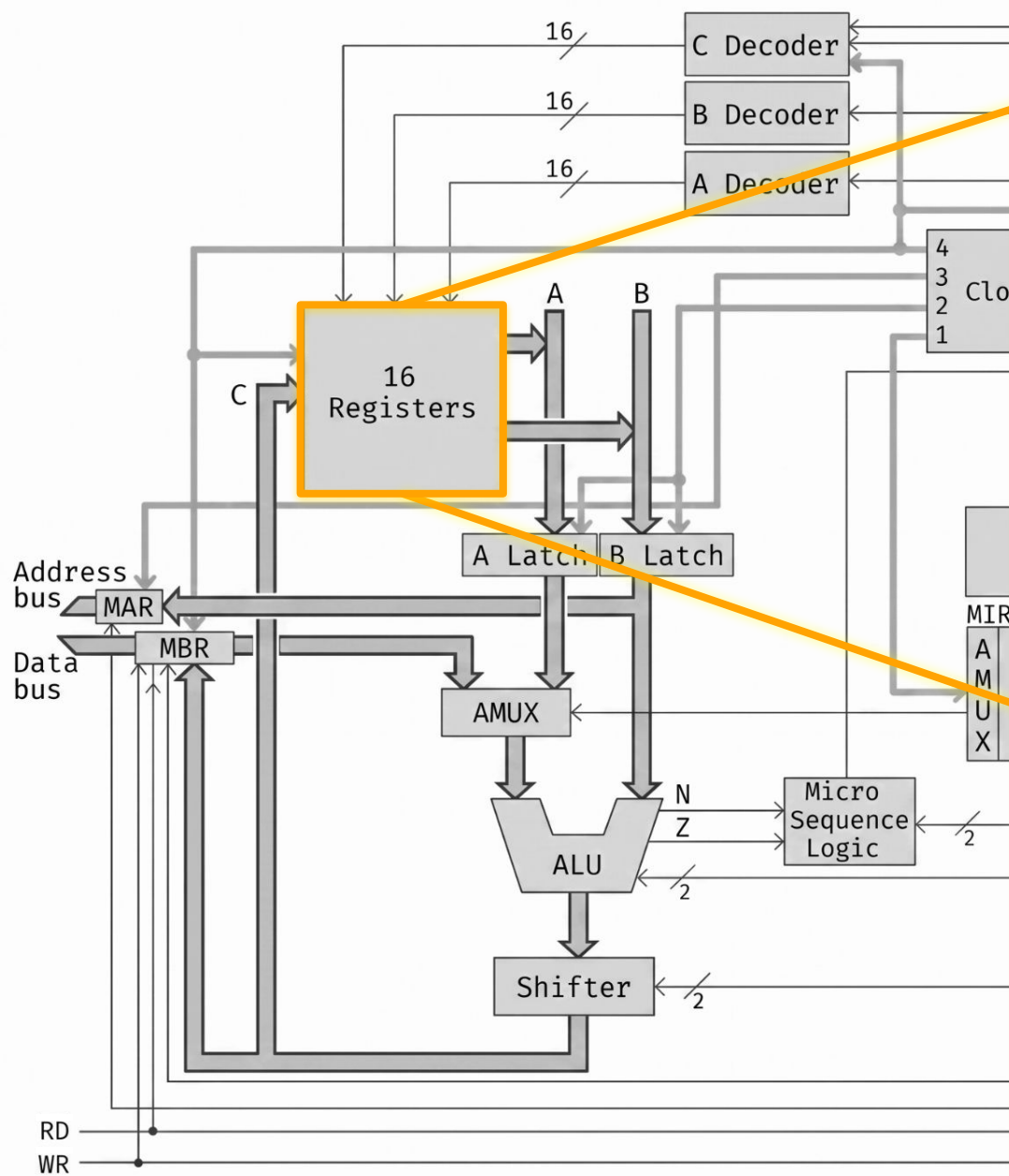
Ejecución directa por el Hardware

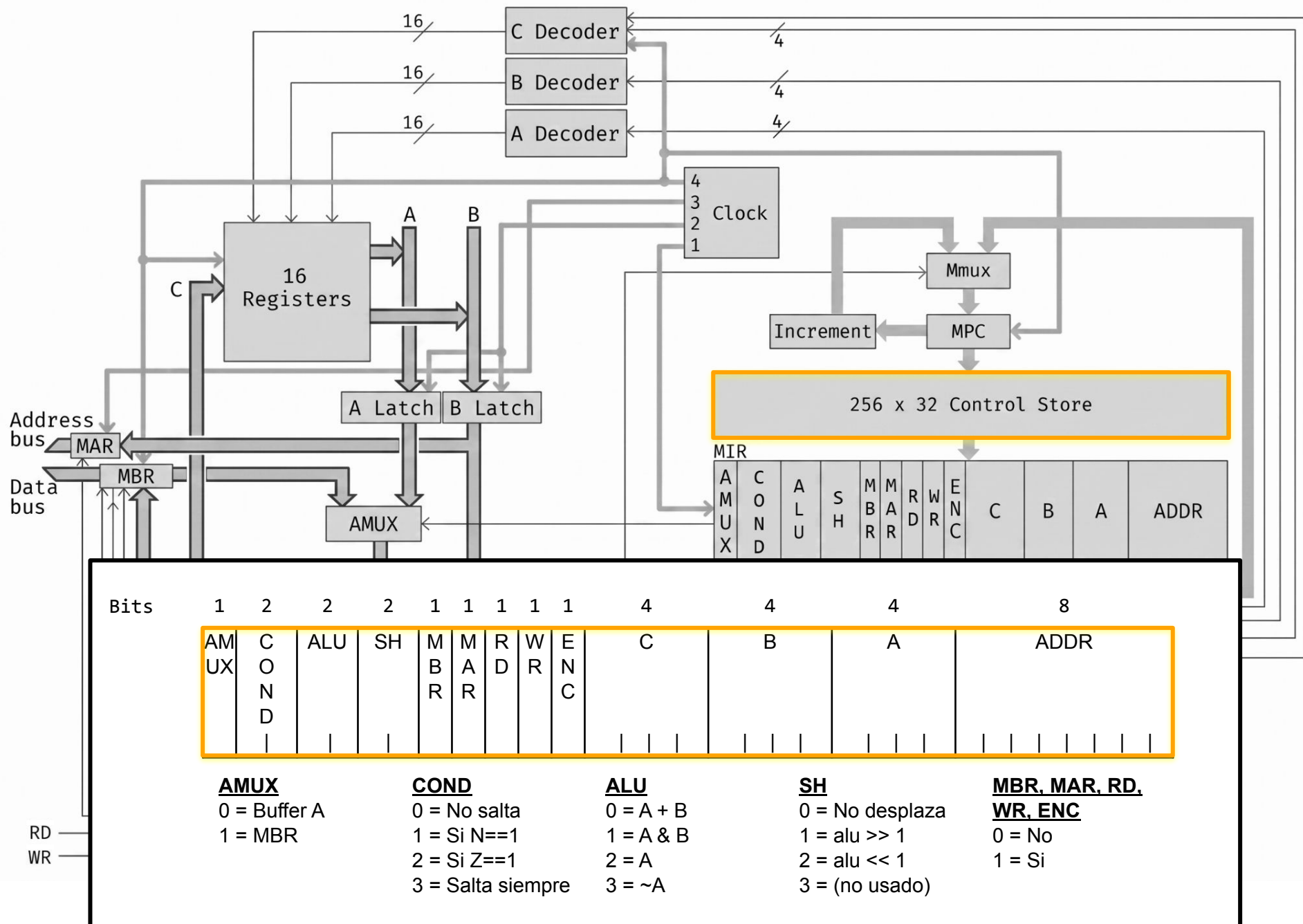
NIVEL 0

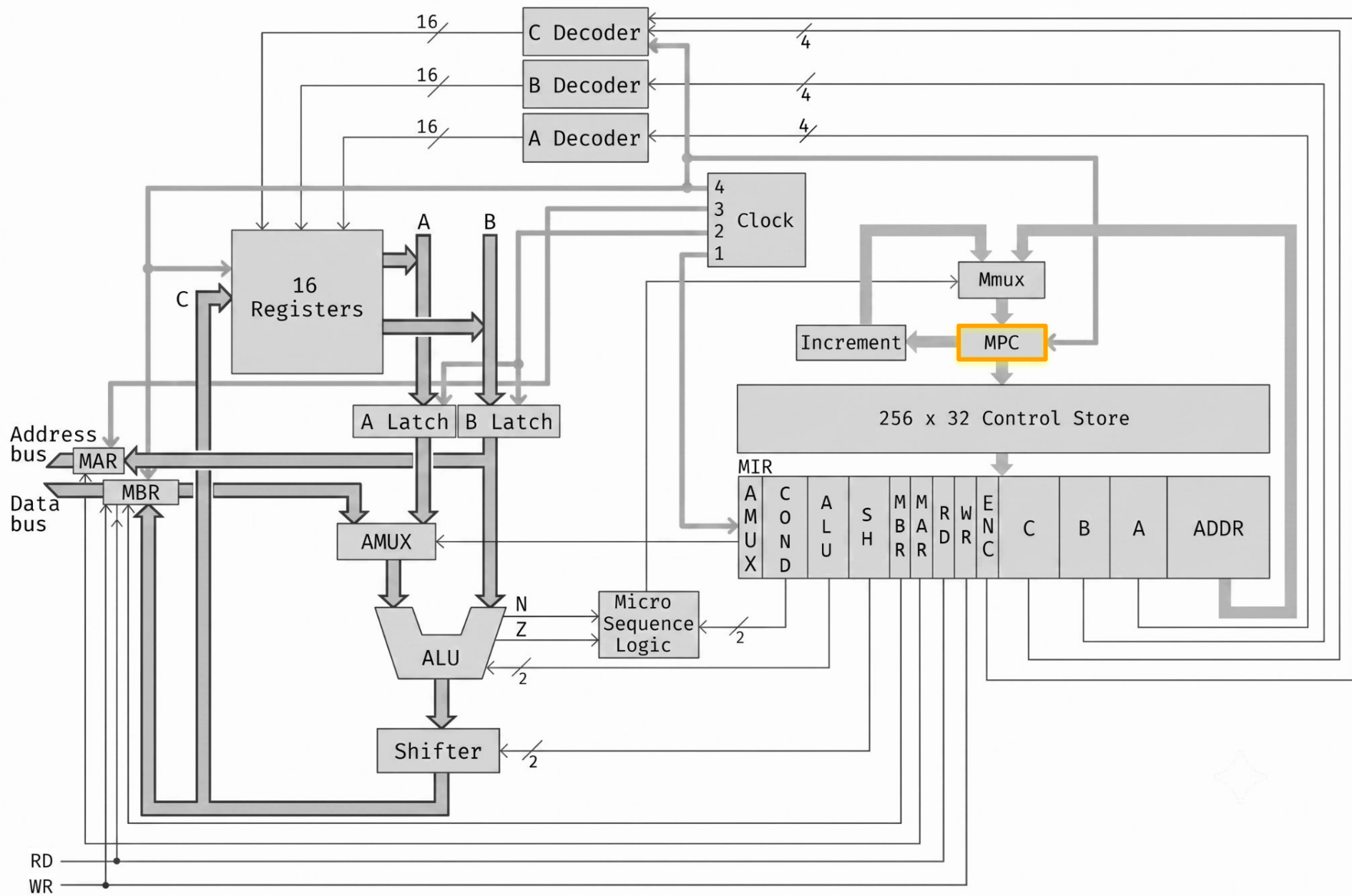
Nivel de lógica digital

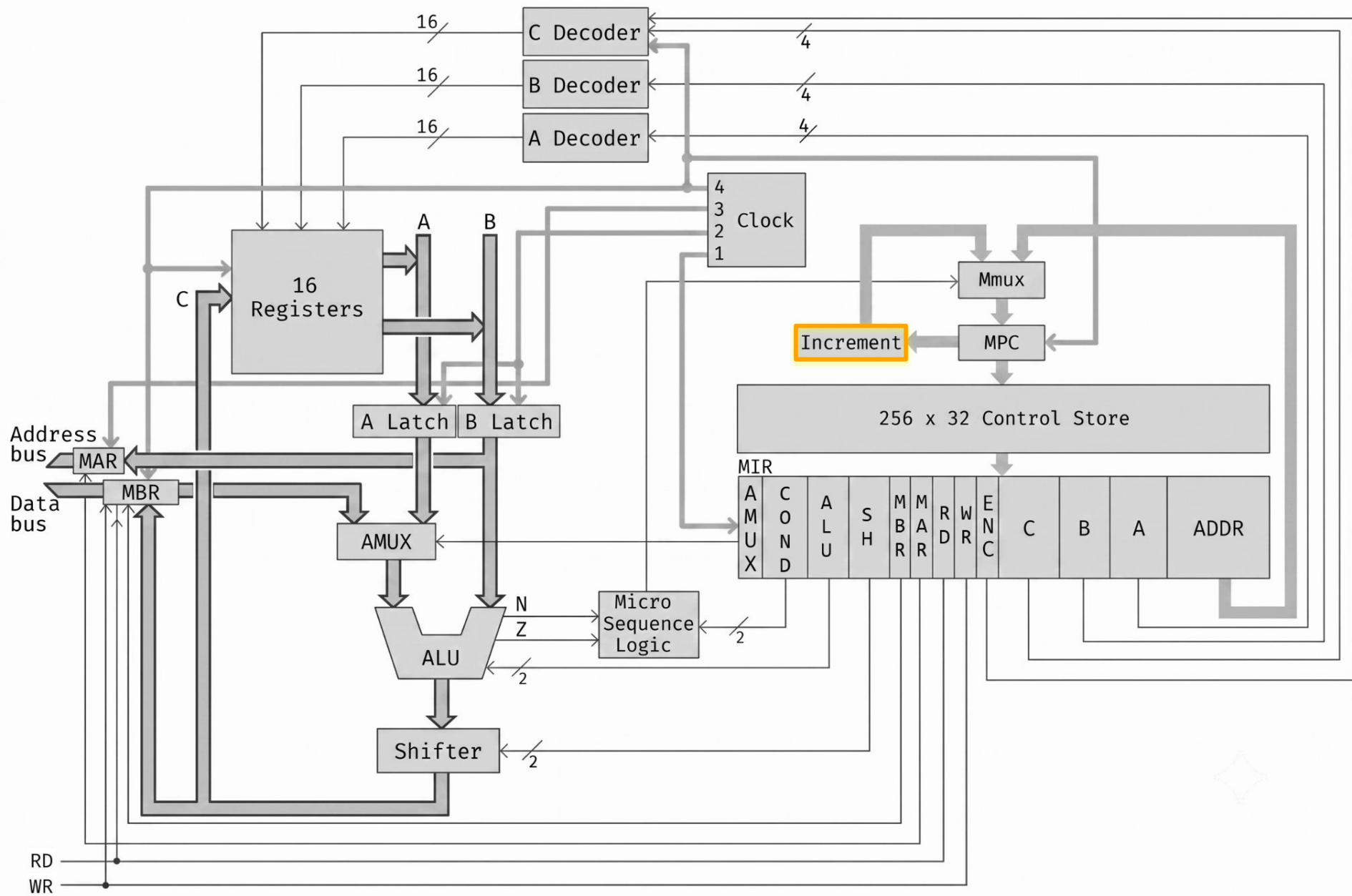


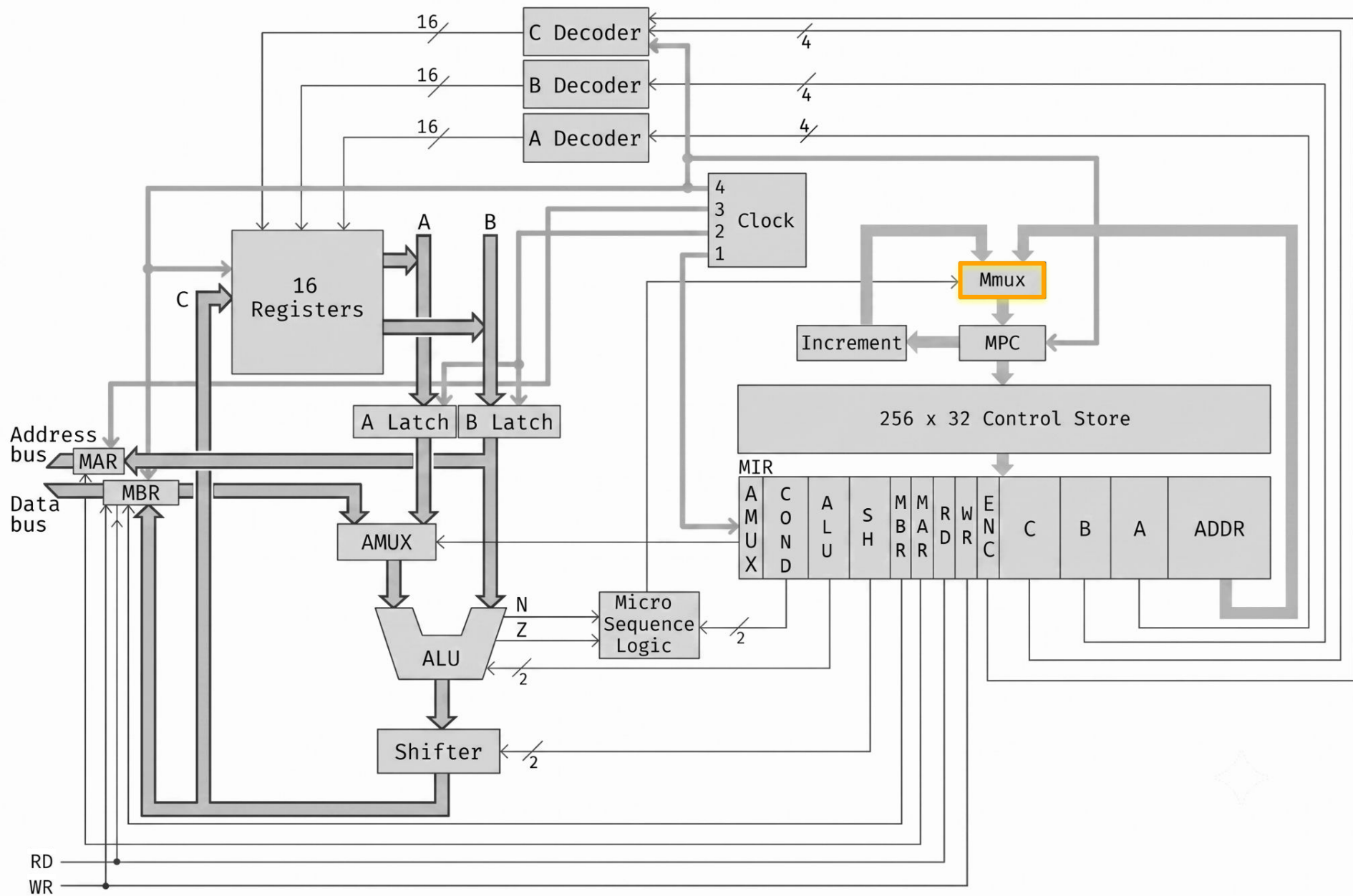


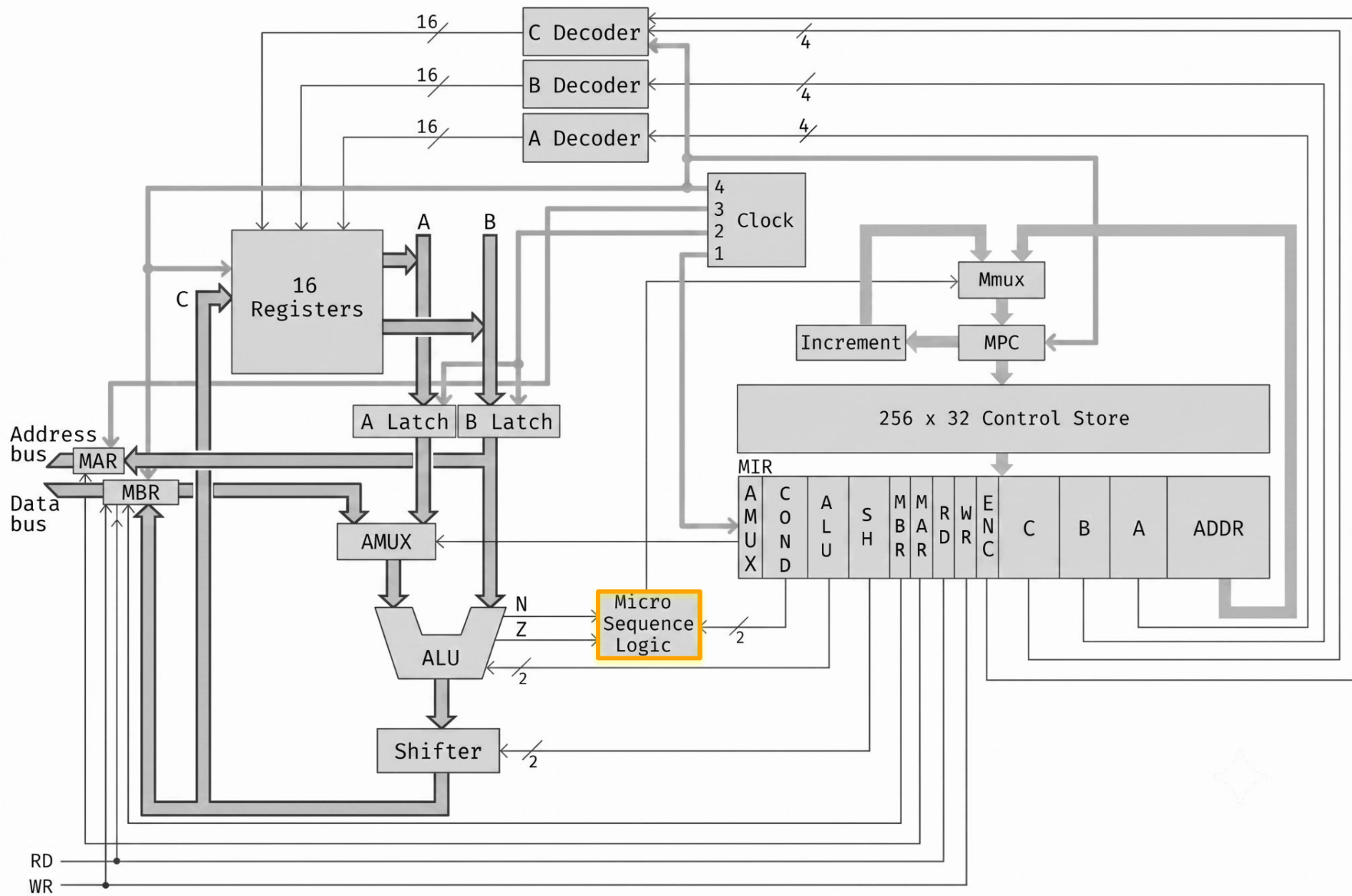












Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac>=0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<=x<=4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:=ac+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

STORE DIRECT

...
STOD 9
...

Guardar el valor del AC en la memoria(9)

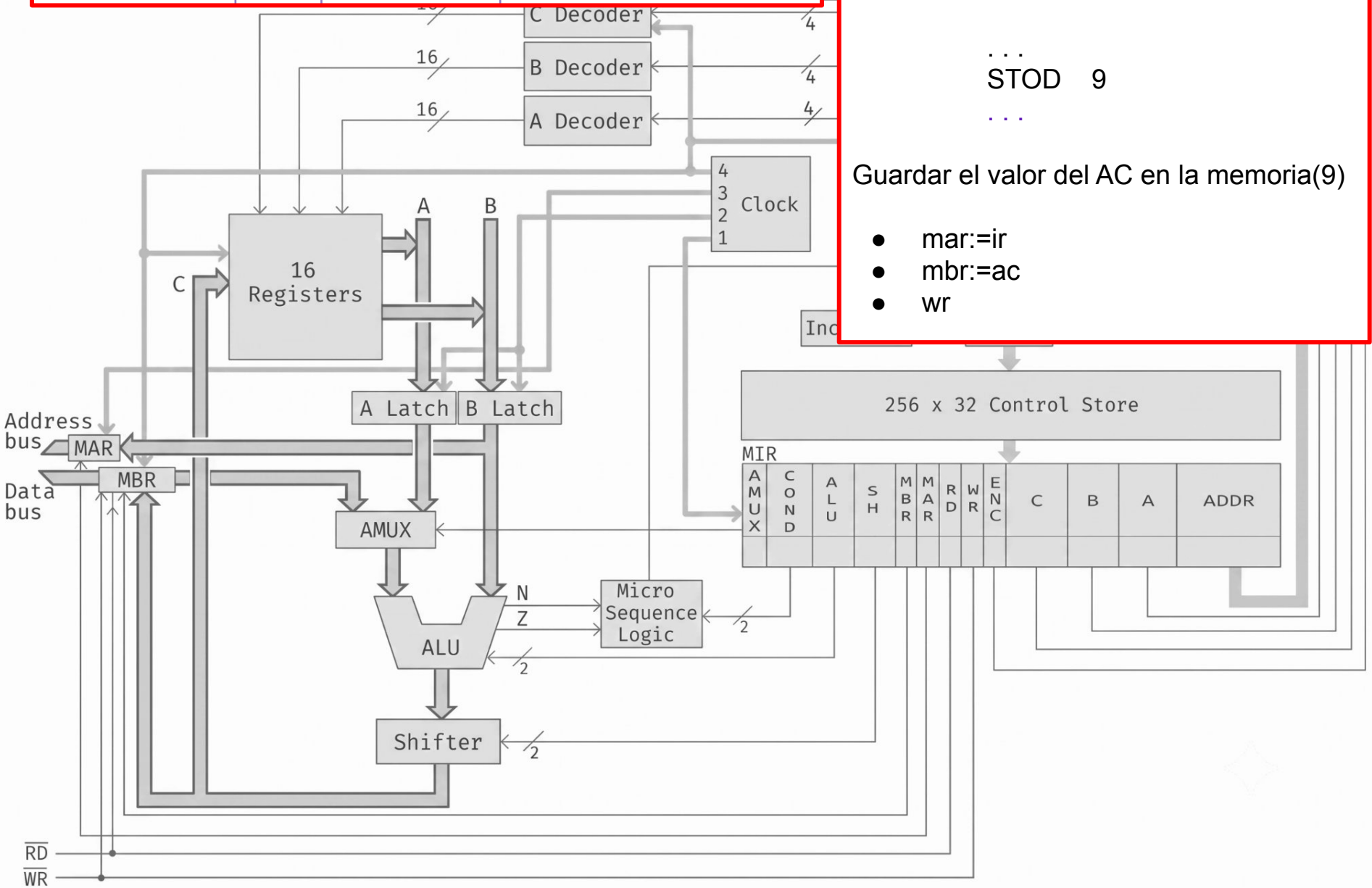
0001xxxxxxxxxxxx STOD Store direct m[x] := ac

STORE DIRECT

...
STOD 9
...

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr



EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; wr ;	0	0	2	0	1	1	0	1	0	0	3	1	0

0001xxxxxxxxxxxx

STOD

Store direct

m[x] := ac

STORE DIRECT

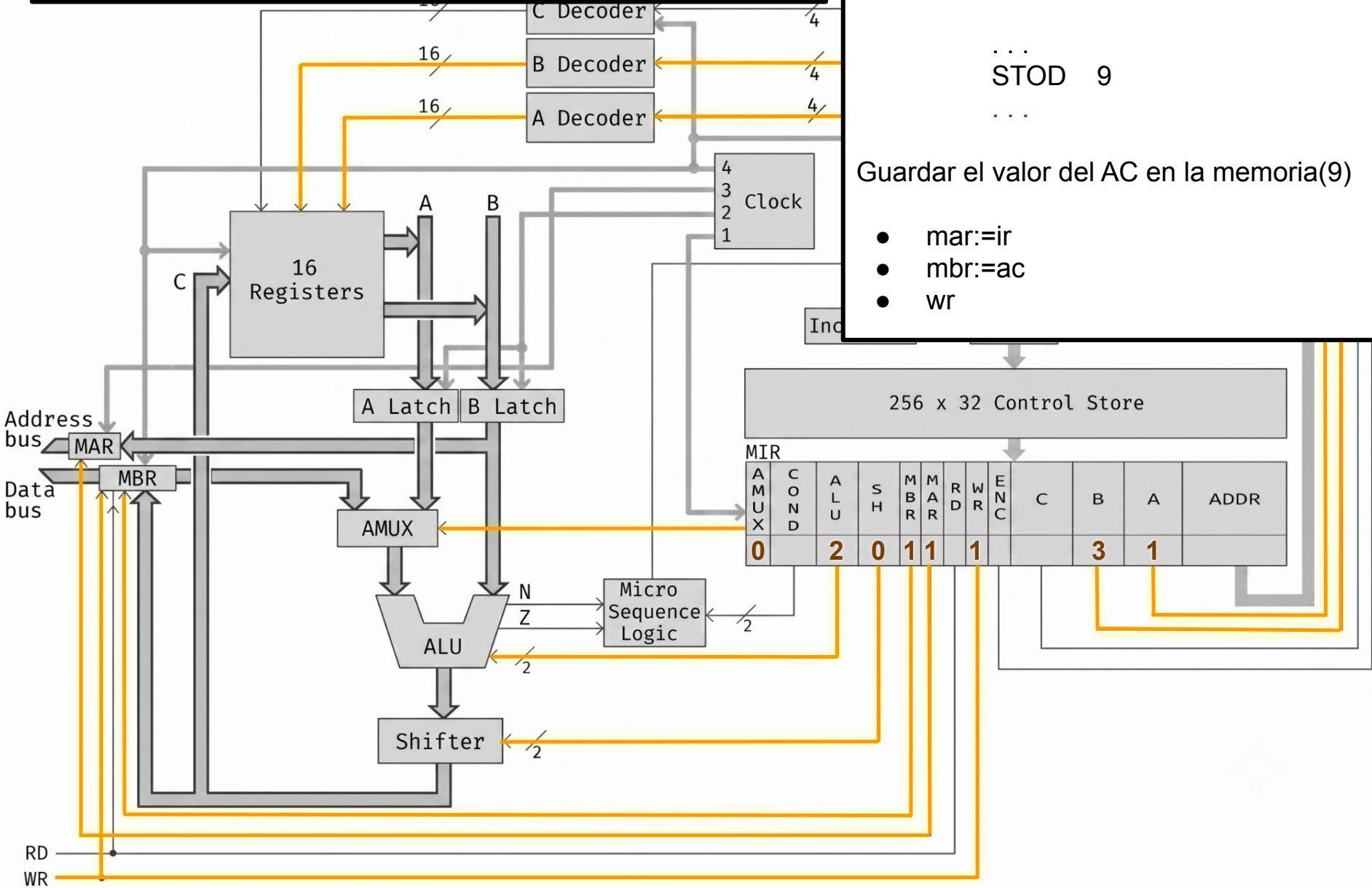
...

STOD 9

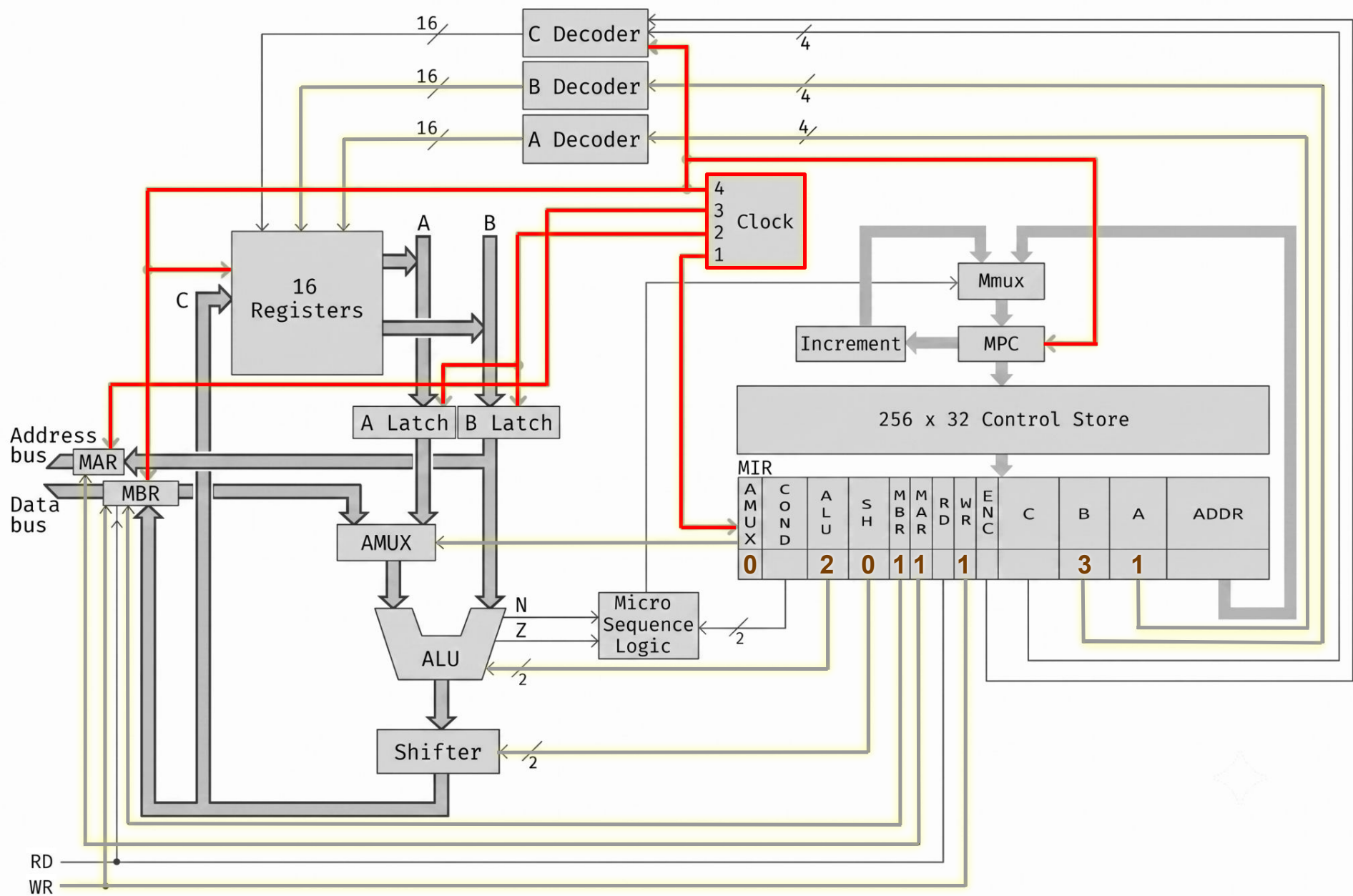
...

Guardar el valor del AC en la memoria(9)

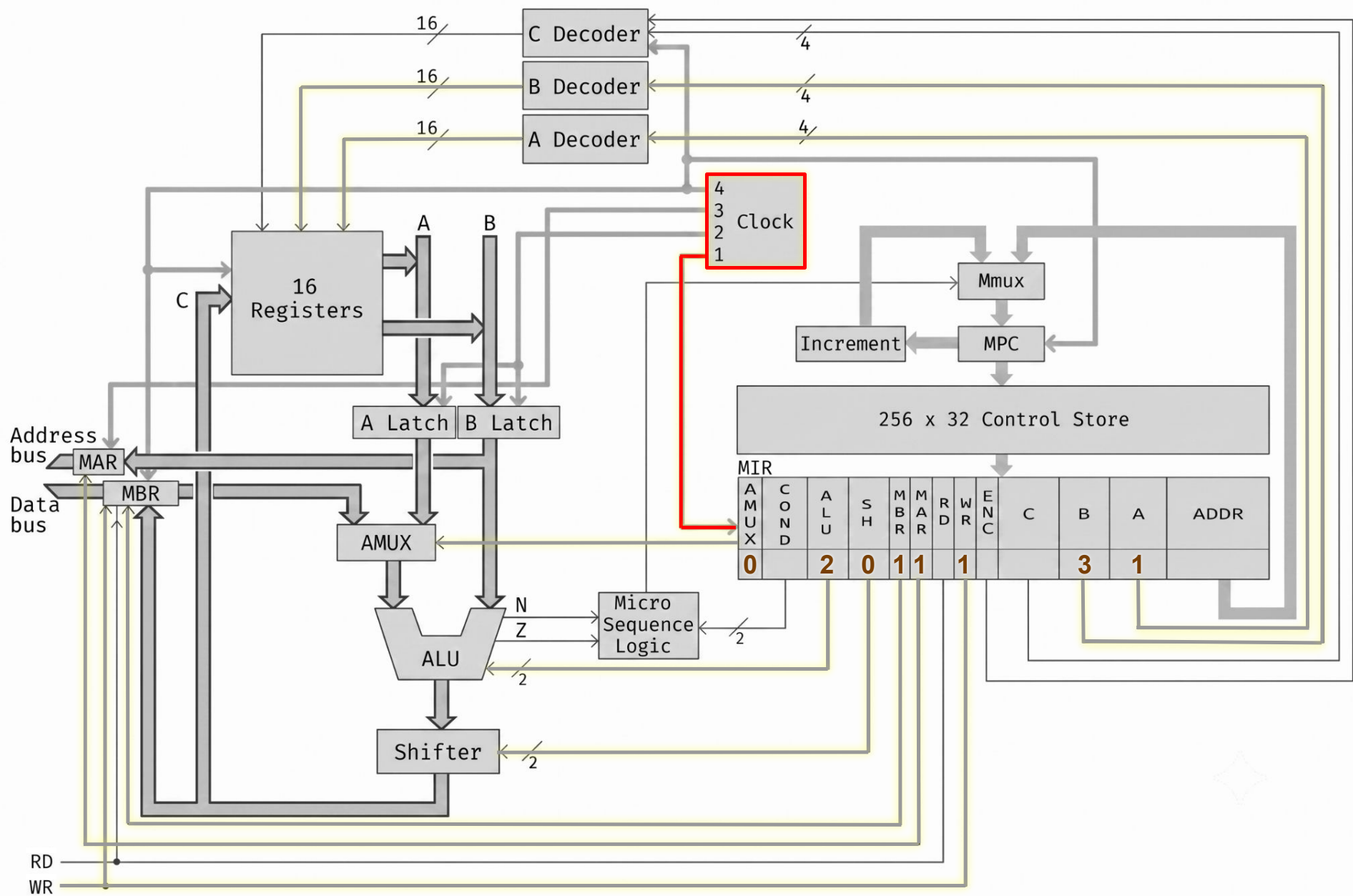
- mar:=ir
- mbr:=ac
- wr



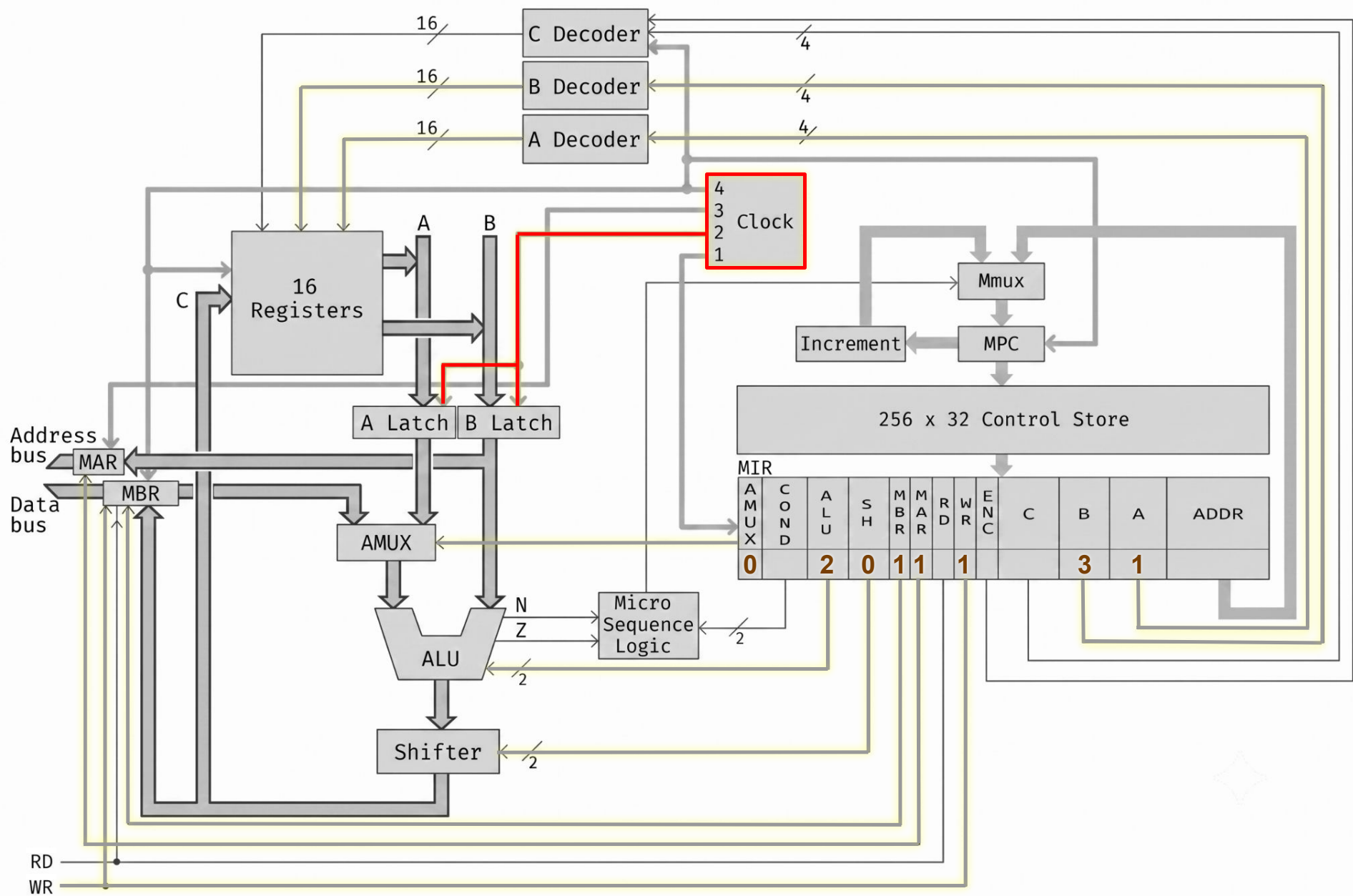
EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; wr ;	0	0	2	0	1	1	0	1	0	0	3	1	0



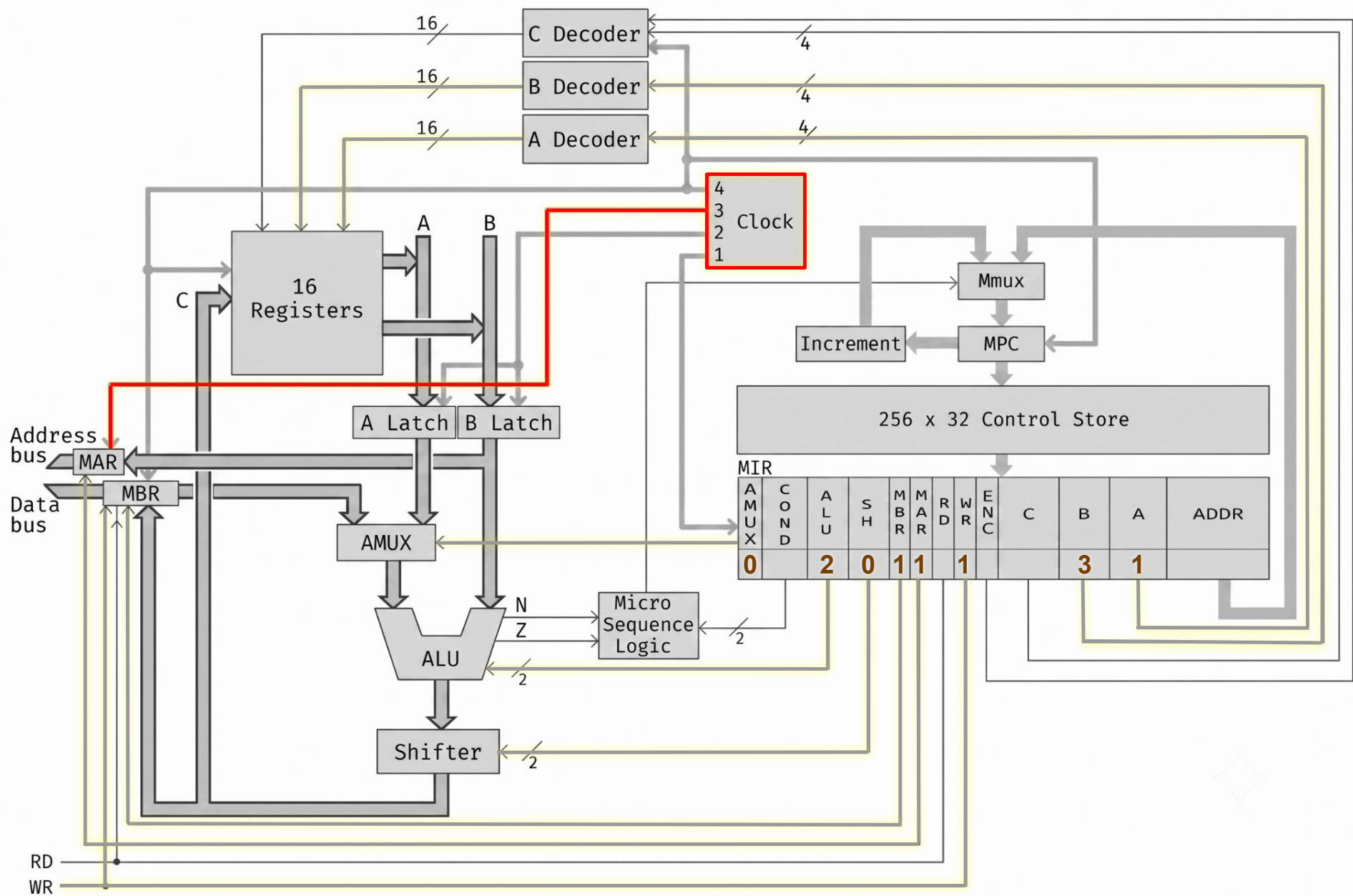
EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; wr ;	0	0	2	0	1	1	0	1	0	0	3	1	0



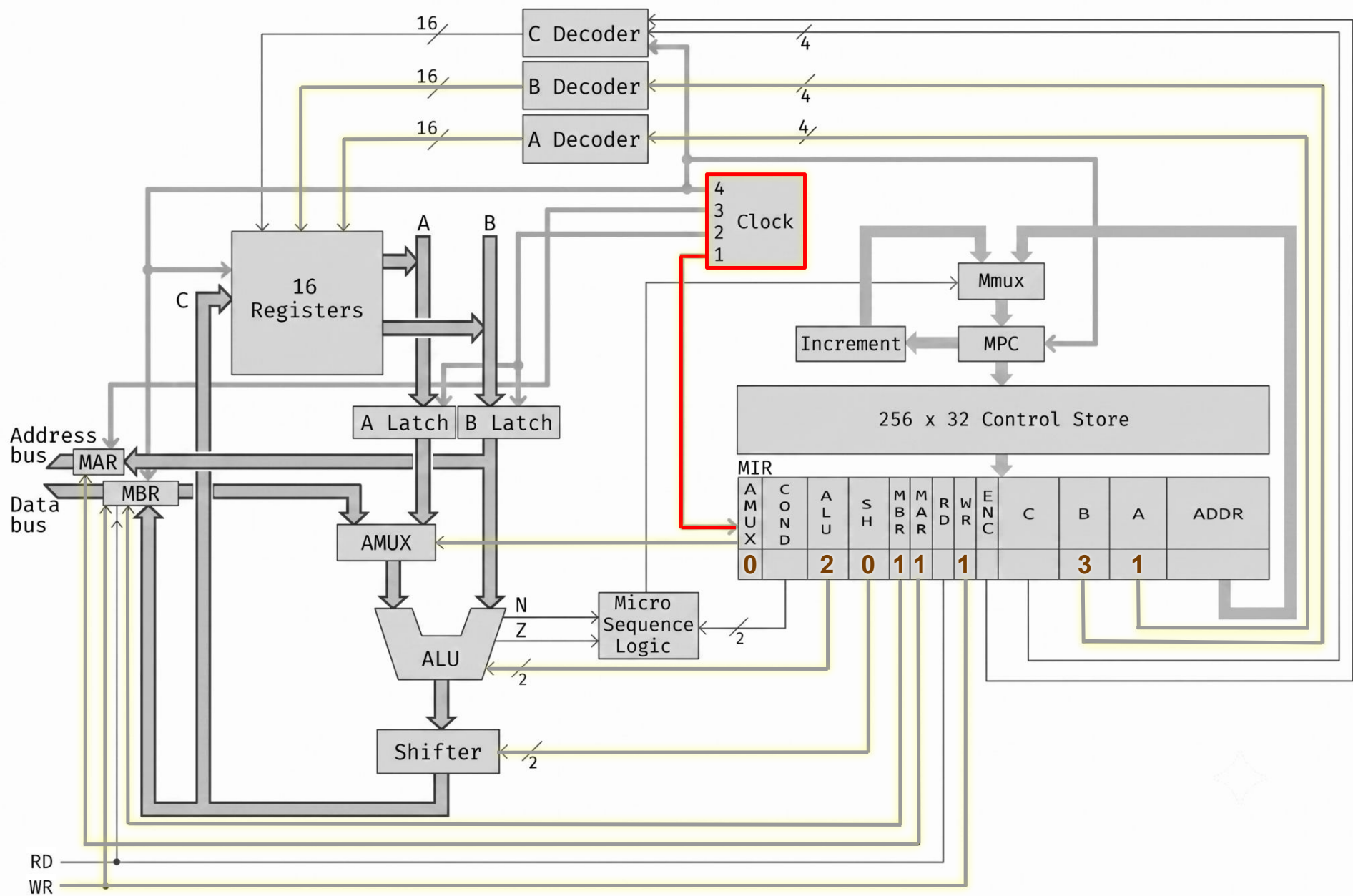
EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; wr ;	0	0	2	0	1	1	0	1	0	0	3	1	0



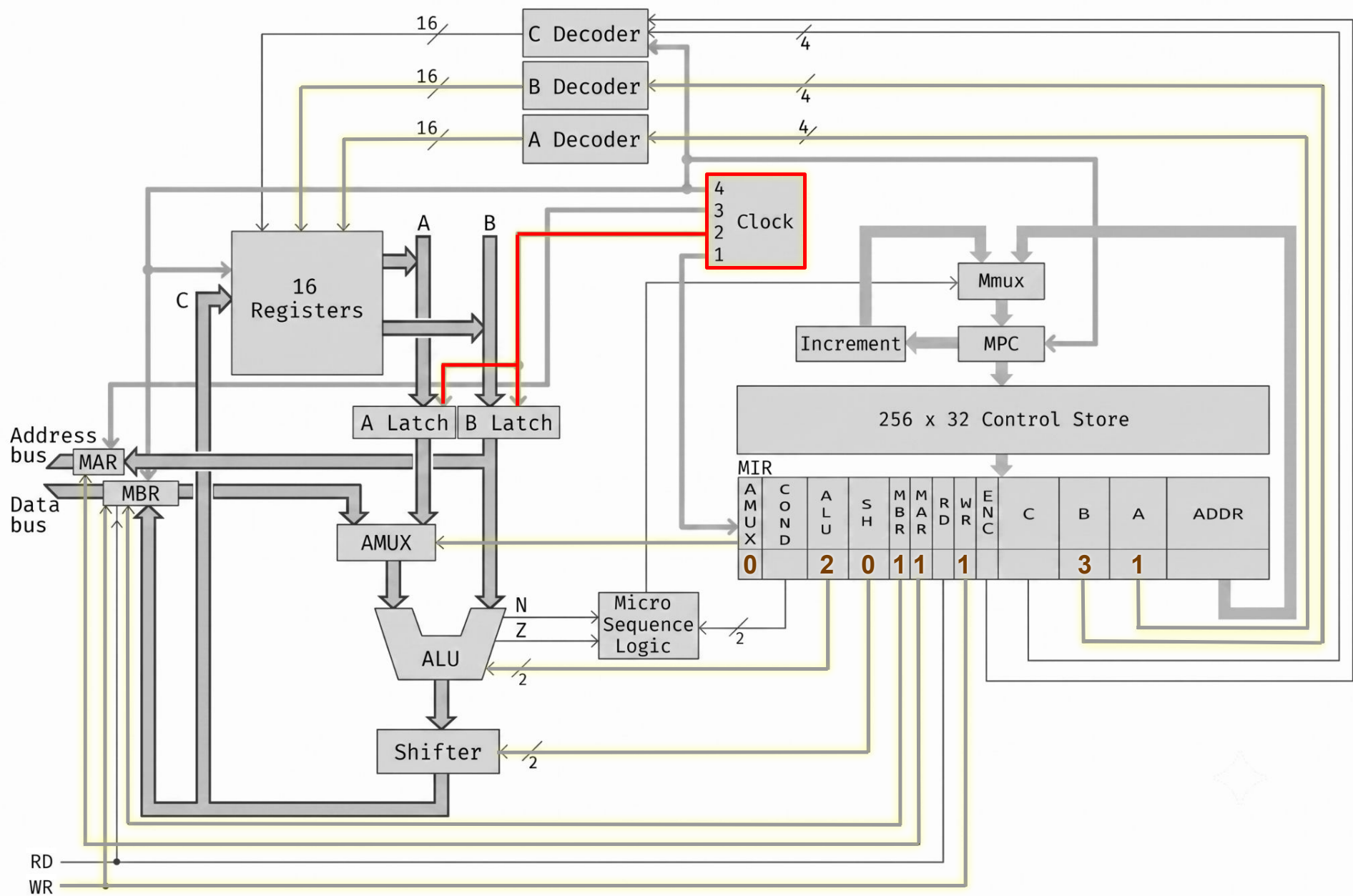
EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; wr ;	0	0	2	0	1	1	0	1	0	0	3	1	0



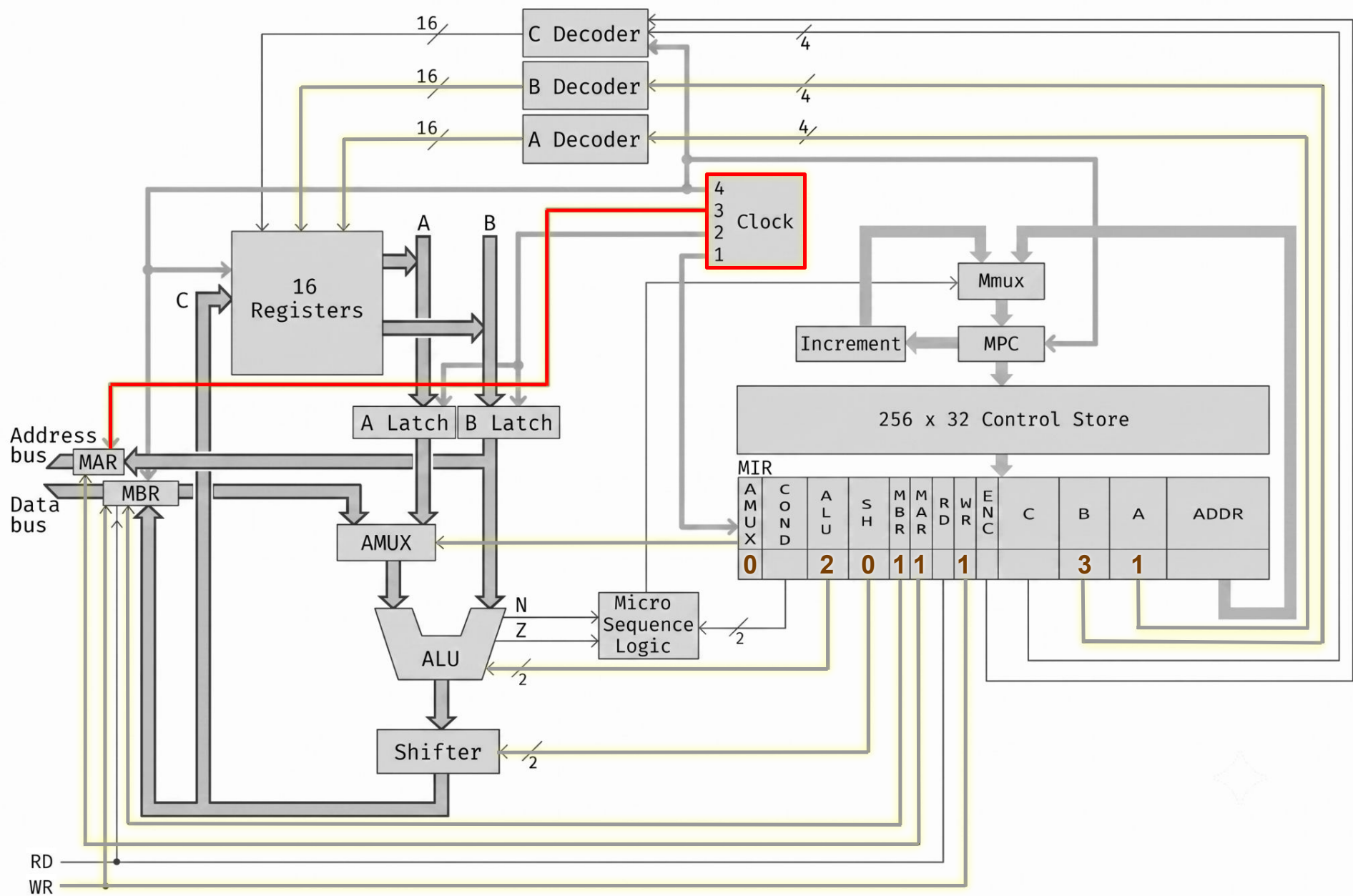
EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; wr ;	0	0	2	0	1	1	0	1	0	0	3	1	0



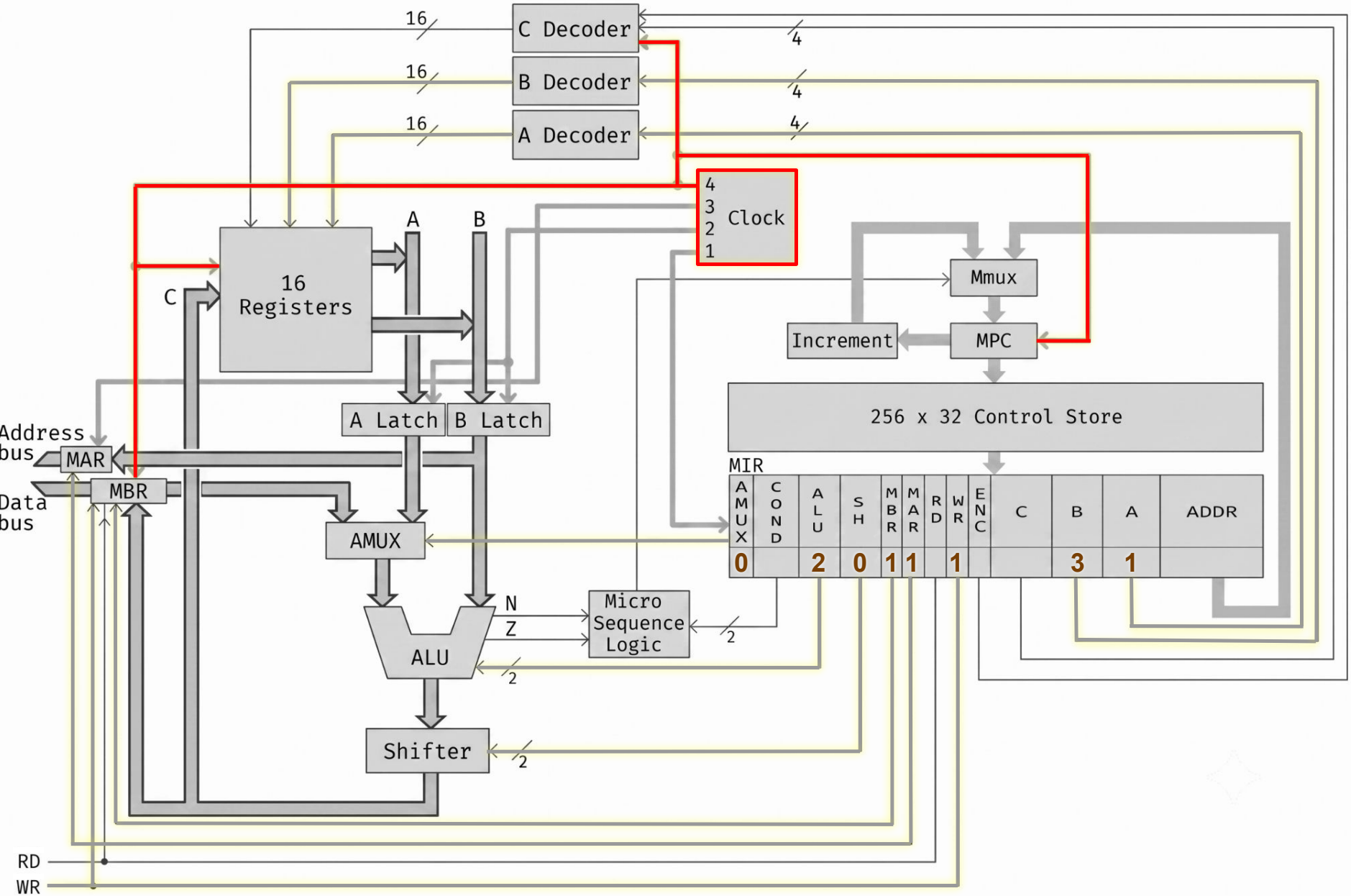
EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; wr ;	0	0	2	0	1	1	0	1	0	0	3	1	0



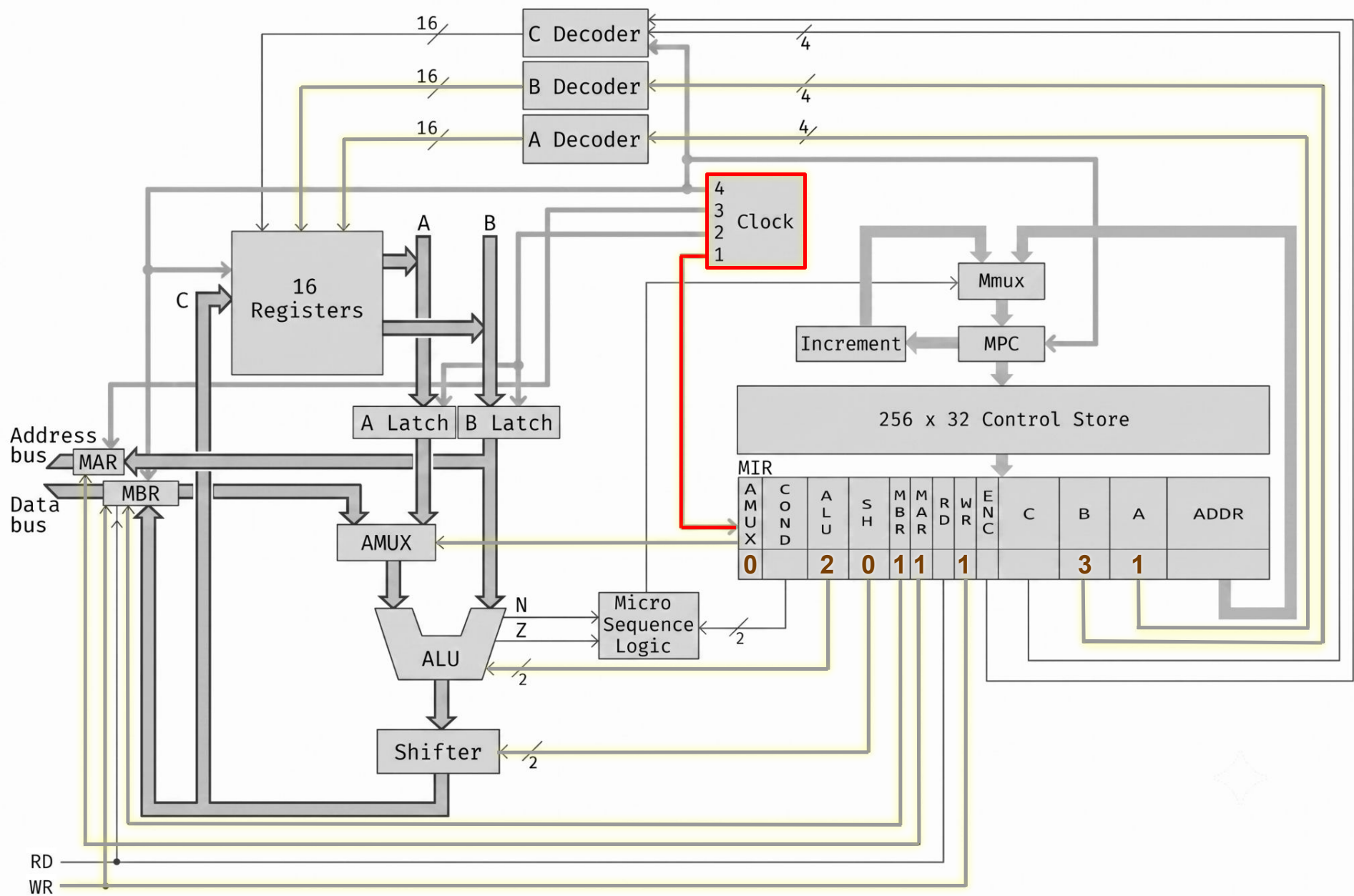
EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; wr ;	0	0	2	0	1	1	0	1	0	0	3	1	0



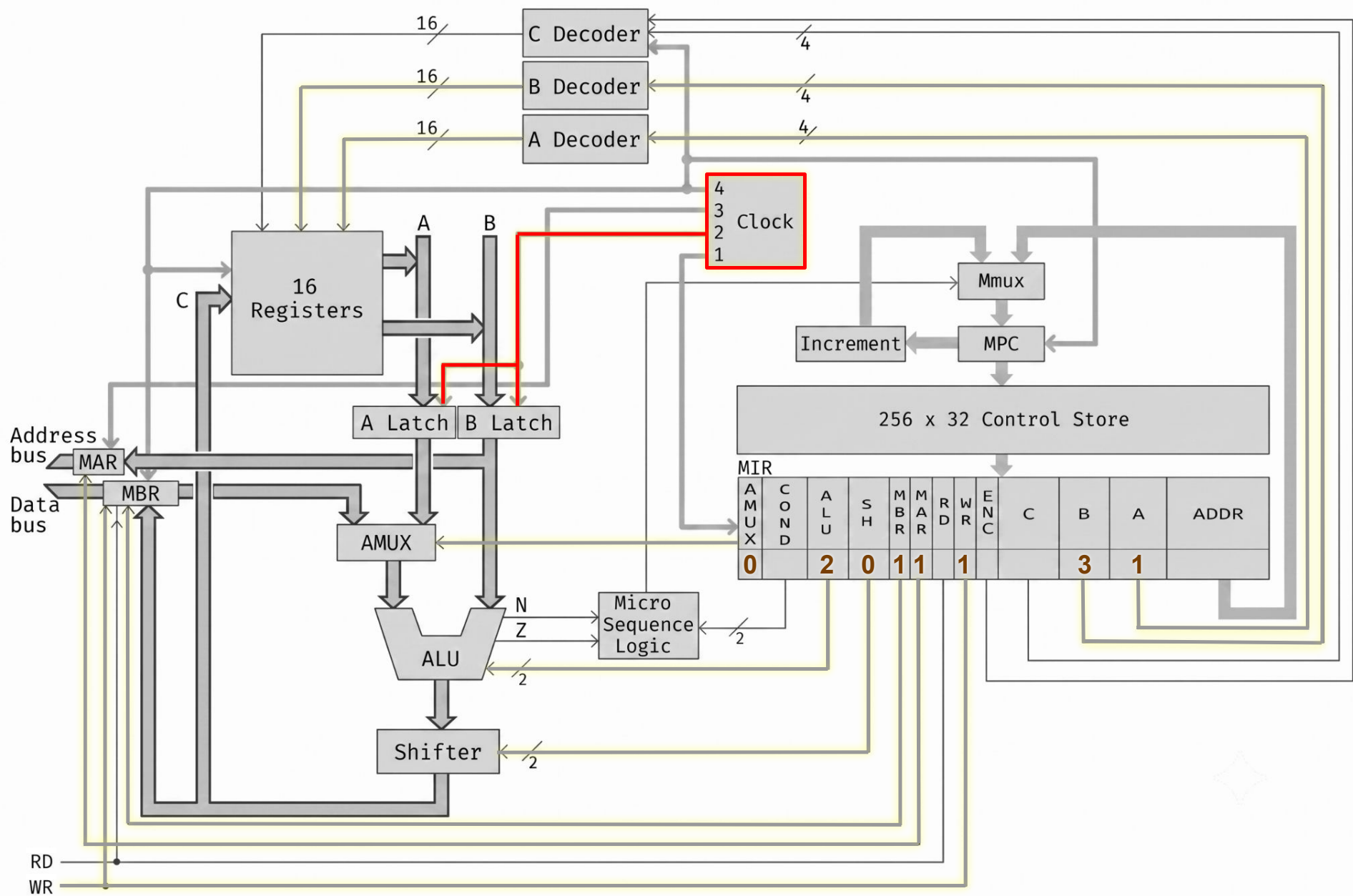
EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; wr ;	0	0	2	0	1	1	0	1	0	0	3	1	0



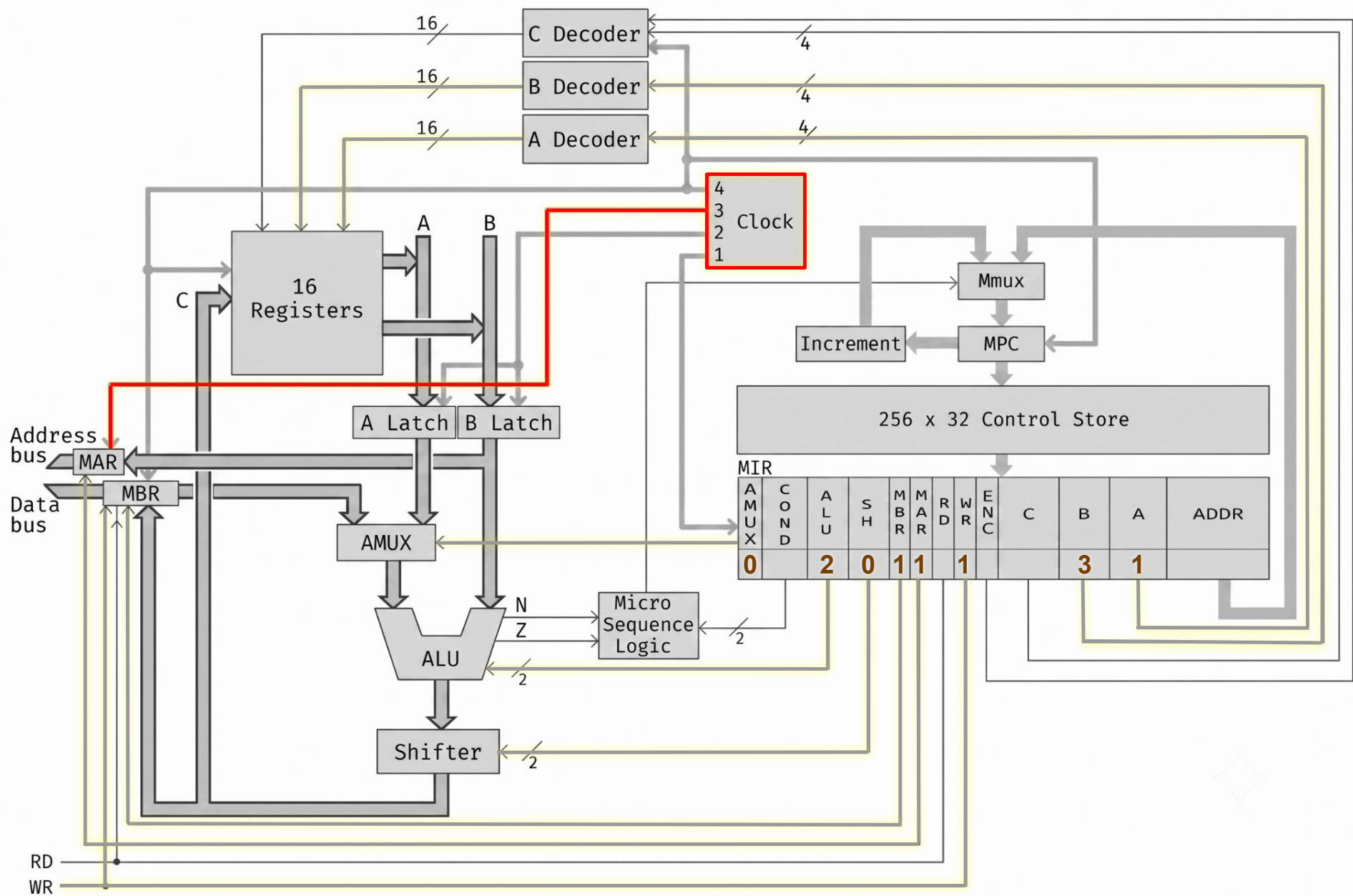
EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; wr ;	0	0	2	0	1	1	0	1	0	0	3	1	0



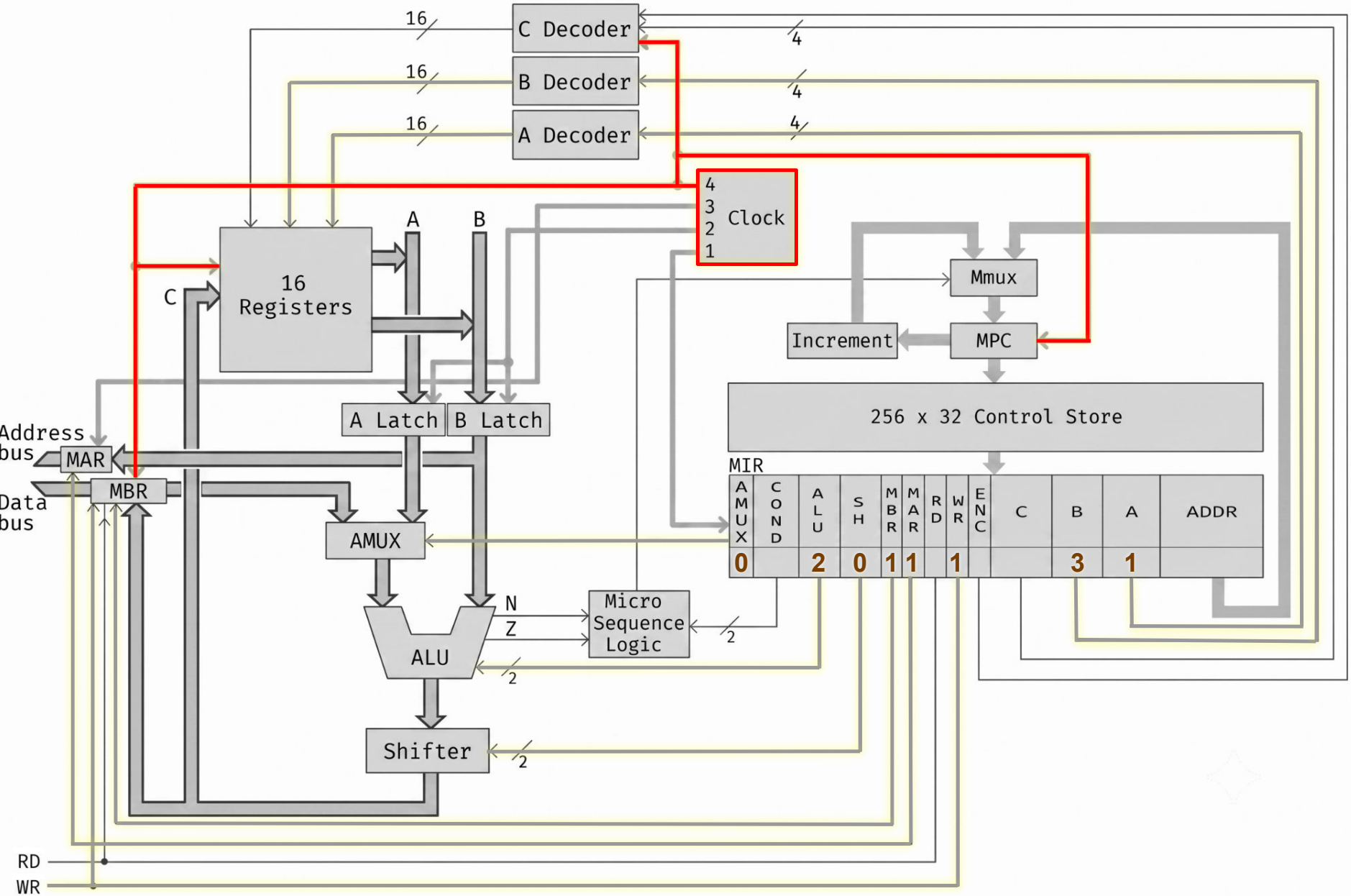
EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; wr ;	0	0	2	0	1	1	0	1	0	0	3	1	0



EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; wr ;	0	0	2	0	1	1	0	1	0	0	3	1	0



EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; wr ;	0	0	2	0	1	1	0	1	0	0	3	1	0



0: mar := pc ; rd ;	{ciclo principal}	40: tir := lshift(tir) ; if n then goto 46 ;	{110x o 111x?}
1: pc := pc + 1 ; rd ;	{incrementa pc}	41: alu := tir ; if n then goto 44 ;	{1100 o 1101?}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}	42: alu := ac ; if n then goto 22 ;	{1100 = JNEG}
3: tir := lshift(ir + ir) ; if n then goto 19 ;		43: goto 0 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}	44: alu := ac ; if z then goto 0 ;	{1101 = JNZE}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}	45: pc := band(ir , amask) ; goto 0 ;	
6: mar := ir ; rd ;	{0000 = LODD}	46: tir := lshift(tir) ; if n then goto 50 ;	
7: rd ;		47: sp := sp + (-1) ;	{1110 = CALL}
8: ac := mbr ; goto 0 ;		48: mar := sp ; mbr := pc ; wr ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}	49: pc := band(ir , amask) ; wr ; goto 0 ;	
10: wr ; goto 0 ;		50: tir := lshift(tir) ; if n then goto 65 ;	{111, examina addr}
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}	51: tir := lshift(tir) ; if n then goto 59 ;	
12: mar := ir ; rd ;	{0010 = ADDD}	52: alu := tir ; if n then goto 56 ;	
13: rd ;		53: mar := ac ; rd ;	{1111000 = PSHI}
14: ac := mbr + ac ; goto 0 ;		54: sp := sp + (-1) ; rd ;	
15: mar := ir ; rd ;	{0011 = SUBD}	55: mar := sp ; wr ; goto 10 ;	
16: ac := ac + 1 ; rd ;	{No se produce el salto}	56: mar := sp ; sp := sp + 1 ; rd ;	{1111001 = POPI}
17: a := inv(mbr) ;		57: rd ;	
18: ac := ac + a ; goto 0 ;		58: mar := ac ; wr ; goto 10 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}	59: alu := tir ; if n then goto 62 ;	
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}	60: sp := sp + (-1) ;	{1111010 = PUSH}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}	61: mar := sp ; mbr := ac ; wr ; goto 10 ;	
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}	62: mar := sp ; sp := sp + 1 ; rd ;	{1111011 = POP}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}	63: rd ;	
24: goto 0 ;	{no se produce el salto}	64: ac := mbr ; goto 0 ;	
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}	65: tir := lshift(tir) ; if n then goto 73 ;	
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}	66: alu := tir ; if n then goto 70 ;	
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}	67: mar := sp ; sp := sp + 1 ; rd ;	{1111100 = RETN}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}	68: rd ;	
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}	69: pc := mbr ; goto 0 ;	
30: alu := tir ; if n then goto 33 ;	{1000 o 1001?}	70: a := ac ;	{1111101 = SWAP}
31: a := ir + sp ;	{1000 = LODL}	71: ac := sp ;	
32: mar := a ; rd ; goto 7 ;		72: sp := a ; goto 0 ;	
33: a := ir + sp ;	{1001 = STOL}	73: alu := tir ; if n then goto 76 ;	
34: mar := a ; mbr := ac ; wr ; goto 10 ;		74: a := band(ir , smask) ;	{1111110 = INSP}
35: alu := tir ; if n then goto 38 ;	{1010 o 1011?}	75: sp := sp + a ; goto 0 ;	
36: a := ir + sp ;	{1010 = ADDL}	76: a := band(ir , smask) ;	{1111111 = DESP}
37: mar := a ; rd ; goto 13 ;		77: a := inv(a) ;	
38: a := ir + sp ;	{1011 = SUBL}	78: a := a + 1 ; goto 75 ;	
39: mar := a ; rd ; goto 16 ;			

Microprograma

0: mar := pc ; rd ;	{ciclo principal}	40: tir := lshift(tir) ; if n then goto 46 ;	{110x o 111x?}
1: pc := pc + 1 ; rd ;	{incrementa pc}	41: alu := tir ; if n then goto 44 ;	{1100 o 1101?}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}	42: alu := ac ; if n then goto 22 ;	{1100 = JNEG}
3: tir := lshift(ir + ir) ; if n then goto 19 ;		43: goto 0 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}	44: alu := ac ; if z then goto 0 ;	{1101 = JNZE}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}	45: pc := band(ir , amask) ; goto 0 ;	
6: mar := ir ; rd ;	{0000 = LODD}	46: tir := lshift(tir) ; if n then goto 50 ;	
7: rd ;		47: sp := sp + (-1) ;	{1110 = CALL}
8: ac := mbr ; goto 0 ;		48: mar := sp ; mbr := pc ; wr ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}	49: pc := band(ir , amask) ; wr ; goto 0 ;	
10: wr ; goto 0 ;		50: tir := lshift(tir) ; if n then goto 65 ;	{111, examina addr}
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}	51: tir := lshift(tir) ; if n then goto 59 ;	
12: mar := ir ; rd ;	{0010 = ADDD}	52: alu := tir ; if n then goto 56 ;	
13: rd ;		53: mar := ac ; rd ;	{1111000 = PSHI}
14: ac := mbr + ac ; goto 0 ;		54: sp := sp + (-1) ; rd ;	
15: mar := ir ; rd ;	{0011 = SUBD}	55: mar := sp ; wr ; goto 10 ;	
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}	56: mar := sp ; sp := sp + 1 ; rd ;	{1111001 = POPI}
17: a := inv(mbr) ;		57: rd ;	
18: ac := ac + a ; goto 0 ;		58: mar := ac ; wr ; goto 10 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}	59: alu := tir ; if n then goto 62 ;	
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}	60: sp := sp + (-1) ;	{1111010 = PUSH}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}	61: mar := sp ; mbr := ac ; wr ; goto 10 ;	
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}	62: mar := sp ; sp := sp + 1 ; rd ;	{1111011 = POP}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}	63: rd ;	
24: goto 0 ;	{no se produce el salto}	64: ac := mbr ; goto 0 ;	
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}	65: tir := lshift(tir) ; if n then goto 73 ;	
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}	66: alu := tir ; if n then goto 70 ;	
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}	67: mar := sp ; sp := sp + 1 ; rd ;	{1111100 = RETN}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}	68: rd ;	
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}	69: pc := mbr ; goto 0 ;	
30: alu := tir ; if n then goto 33 ;	{1000 o 1001?}	70: a := ac ;	{1111101 = SWAP}
31: a := ir + sp ;	{1000 = LODL}	71: ac := sp ;	
32: mar := a ; rd ; goto 7 ;		72: sp := a ; goto 0 ;	
33: a := ir + sp ;	{1001 = STOL}	73: alu := tir ; if n then goto 76 ;	
34: mar := a ; mbr := ac ; wr ; goto 10 ;		74: a := band(ir , smask) ;	{1111110 = INSP}
35: alu := tir ; if n then goto 38 ;	{1010 o 1011?}	75: sp := sp + a ; goto 0 ;	
36: a := ir + sp ;	{1010 = ADDL}	76: a := band(ir , smask) ;	{1111111 = DESP}
37: mar := a ; rd ; goto 13 ;		77: a := inv(a) ;	
38: a := ir + sp ;	{1011 = SUBL}	78: a := a + 1 ; goto 75 ;	
39: mar := a ; rd ; goto 16 ;			

EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; wr ;	0	0	2	0	1	1	0	1	0	0	3	1	0

0001xxxxxxxxxxxx	STOD	Store direct	m[x] := ac
------------------	------	--------------	------------

STORE DIRECT

8: ac := mbr ; goto 0 ;		48: mar := sp ; mbr := pc ; wr ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}	49: pc := band(ir , amask) ; wr ; goto 0 ;	
10: wr ; goto 0 ;		50: tir := lshift(tir) ; if n then goto 65 ;	{111, examina addr}
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}	51: tir := lshift(tir) ; if n then goto 59 ;	
12: mar := ir ; rd ;	{0010 = ADDD}	52: alu := tir ; if n then goto 56 ;	
13: rd ;		53: mar := ac ; rd ;	{1111000 = PSHI}
14: ac := mbr + ac ; goto 0 ;		54: sp := sp + (-1) ; rd ;	
15: mar := ir ; rd ;	{0011 = SUBD}	55: mar := sp ; wr ; goto 10 ;	
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}	56: mar := sp ; sp := sp + 1 ; rd ;	{1111001 = POPI}
17: a := inv(mbr) ;		57: rd ;	
18: ac := ac + a ; goto 0 ;		58: mar := ac ; wr ; goto 10 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}	59: alu := tir ; if n then goto 62 ;	
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}	60: sp := sp + (-1) ;	{1111010 = PUSH}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}	61: mar := sp ; mbr := ac ; wr ; goto 10 ;	
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}	62: mar := sp ; sp := sp + 1 ; rd ;	{1111011 = POP}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}	63: rd ;	
24: goto 0 ;	{no se produce el salto}	64: ac := mbr ; goto 0 ;	
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}	65: tir := lshift(tir) ; if n then goto 73 ;	
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}	66: alu := tir ; if n then goto 70 ;	
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}	67: mar := sp ; sp := sp + 1 ; rd ;	{1111100 = RETN}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}	68: rd ;	
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}	69: pc := mbr ; goto 0 ;	
30: alu := tir ; if n then goto 33 ;	{1000 o 1001?}	70: a := ac ;	{1111101 = SWAP}
31: a := ir + sp ;	{1000 = LODL}	71: ac := sp ;	
32: mar := a ; rd ; goto 7 ;		72: sp := a ; goto 0 ;	
33: a := ir + sp ;	{1001 = STOL}	73: alu := tir ; if n then goto 76 ;	
34: mar := a ; mbr := ac ; wr ; goto 10 ;		74: a := band(ir , smask) ;	{1111110 = INSP}
35: alu := tir ; if n then goto 38 ;	{1010 o 1011?}	75: sp := sp + a ; goto 0 ;	
36: a := ir + sp ;	{1010 = ADDL}	76: a := band(ir , smask) ;	{1111111 = DESP}
37: mar := a ; rd ; goto 13 ;		77: a := inv(a) ;	
38: a := ir + sp ;	{1011 = SUBL}	78: a := a + 1 ; goto 75 ;	
39: mar := a ; rd ; goto 16 ;			

EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
wr ; goto 0 ;	0	<u>3</u>	0	0	0	0	0	1	0	0	0	0	<u>0</u>

0001xxxxxxxxxxxx	STOD	Store direct	$m[x] := ac$
------------------	------	--------------	--------------

STORE DIRECT

8: ac := mbr ; goto 0 ;		48: mar := sp ; mbr := pc ; wr ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}	49: pc := band(ir , amask) ; wr ; goto 0 ;	
10: wr ; goto 0 ;		50: tir := lshift(tir) ; if n then goto 65 ;	{111, examina addr}
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}	51: tir := lshift(tir) ; if n then goto 59 ;	
12: mar := ir ; rd ;	{0010 = ADDD}	52: alu := tir ; if n then goto 56 ;	
13: rd ;		53: mar := ac ; rd ;	{1111000 = PSHI}
14: ac := mbr + ac ; goto 0 ;		54: sp := sp + (-1) ; rd ;	
15: mar := ir ; rd ;	{0011 = SUBD}	55: mar := sp ; wr ; goto 10 ;	
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}	56: mar := sp ; sp := sp + 1 ; rd ;	{1111001 = POPI}
17: a := inv(mbr) ;		57: rd ;	
18: ac := ac + a ; goto 0 ;		58: mar := ac ; wr ; goto 10 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}	59: alu := tir ; if n then goto 62 ;	
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}	60: sp := sp + (-1) ;	{1111010 = PUSH}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}	61: mar := sp ; mbr := ac ; wr ; goto 10 ;	
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}	62: mar := sp ; sp := sp + 1 ; rd ;	{1111011 = POP}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}	63: rd ;	
24: goto 0 ;	{no se produce el salto}	64: ac := mbr ; goto 0 ;	
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}	65: tir := lshift(tir) ; if n then goto 73 ;	
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}	66: alu := tir ; if n then goto 70 ;	
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}	67: mar := sp ; sp := sp + 1 ; rd ;	{1111100 = RETN}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}	68: rd ;	
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}	69: pc := mbr ; goto 0 ;	
30: alu := tir ; if n then goto 33 ;	{1000 o 1001?}	70: a := ac ;	{1111101 = SWAP}
31: a := ir + sp ;	{1000 = LODL}	71: ac := sp ;	
32: mar := a ; rd ; goto 7 ;		72: sp := a ; goto 0 ;	
33: a := ir + sp ;	{1001 = STOL}	73: alu := tir ; if n then goto 76 ;	
34: mar := a ; mbr := ac ; wr ; goto 10 ;		74: a := band(ir , smask) ;	{1111110 = INSP}
35: alu := tir ; if n then goto 38 ;	{1010 o 1011?}	75: sp := sp + a ; goto 0 ;	
36: a := ir + sp ;	{1010 = ADDL}	76: a := band(ir , smask) ;	{1111111 = DESP}
37: mar := a ; rd ; goto 13 ;		77: a := inv(a) ;	
38: a := ir + sp ;	{1011 = SUBL}	78: a := a + 1 ; goto 75 ;	
39: mar := a ; rd ; goto 16 ;			

0: mar := pc ; rd ;	{ciclo principal}	Binario	Mnem	Instrucción	Significado
1: pc := pc + 1 ; rd ;	{incrementa pc}	0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}	0001xxxxxxxxxxxx	STOD	Store direct	m[x] :=ac
3: tir := lshift(ir + ir) ; if n then goto 19 ;		0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}	0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}	0100xxxxxxxxxxxx	JPOS	Jump positive	if ac>=0 then pc:=x
6: mar := ir ; rd ;	{0000 = LODD}	0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
7: rd ;		0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
8: ac := mbr ; goto 0 ;		0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<=x<=4095)
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}	1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
10: wr ; goto 0 ;		1001xxxxxxxxxxxx	STOL	Store local	m[x+sp] :=ac
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}	1010xxxxxxxxxxxx	ADDL	Add local	ac:=ac+m[sp+x]
12: mar := ir ; rd ;	{0010 = ADDD}	1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
13: rd ;		1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
14: ac := mbr + ac ; goto 0 ;		1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
15: mar := ir ; rd ;	{0011 = SUBD}	1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp] :=pc; pc:=x
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}	1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp] :=m[ac]
17: a := inv(mbr) ;		1111001000000000	POPI	Pop indirect	m[ac] :=m[sp]; sp:=sp+1
18: ac := ac + a ; goto 0 ;		1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp] :=ac
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}	1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}	1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}	1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}	11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}	11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)
24: goto 0 ;	{no se produce el salto}				
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}				
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}				
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}				
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}				
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}				
30: alu := tir ; if n then goto 33 ;	{1000 o 1001?}				
31: a := ir + sp ;	{1000 = LODL}				
32: mar := a ; rd ; goto 7 ;					
33: a := ir + sp ;	{1001 = STOL}				
34: mar := a ; mbr := ac ; wr ; goto 10 ;					
35: alu := tir ; if n then goto 38 ;	{1010 o 1011?}				
36: a := ir + sp ;	{1010 = ADDL}				
37: mar := a ; rd ; goto 13 ;					
38: a := ir + sp ;	{1011 = SUBL}				
39: mar := a ; rd ; goto 16 ;					

Binario	Mnem	Instrucción	Significado	
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]	40: tir := <i>lshift</i> (tir) ; if n then goto 46 ; {110x o 111x?}
0001xxxxxxxxxxxx	STOD	Store direct	m[x] :=ac	41: alu := tir ; if n then goto 44 ; {1100 o 1101?}
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]	42: alu := ac ; if n then goto 22 ; {1100 = JNEG}
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]	43: goto 0 ;
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac>=0 then pc:=x	44: alu := ac ; if z then goto 0 ; {1101 = JNZE}
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x	45: pc := <i>band</i> (ir , amask) ; goto 0 ;
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x	46: tir := <i>lshift</i> (tir) ; if n then goto 50 ;
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<=x<=4095)	47: sp := sp + (-1) ; {1110 = CALL}
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]	48: mar := sp ; mbr := pc ; wr ;
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp] :=ac	49: pc := <i>band</i> (ir , amask) ; wr ; goto 0 ;
1010xxxxxxxxxxxx	ADDL	Add local	ac:=ac+m[sp+x]	50: tir := <i>lshift</i> (tir) ; if n then goto 65 ; {111, examina addr}
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]	51: tir := <i>lshift</i> (tir) ; if n then goto 59 ;
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x	52: alu := tir ; if n then goto 56 ;
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x	53: mar := ac ; rd ; {1111000 = PSHI}
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x	54: sp := sp + (-1) ; rd ;
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]	55: mar := sp ; wr ; goto 10 ;
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp] ; sp:=sp+1	56: mar := sp ; sp := sp + 1 ; rd ; {1111001 = POPI}
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac	57: rd ;
1111011000000000	POP	Pop from stack	ac:=m[sp] ; sp:=sp+ 1	58: mar := ac ; wr ; goto 10 ;
1111100000000000	RETN	Return	pc:=m[sp] ; sp:=sp+ 1	59: alu := tir ; if n then goto 62 ;
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp	60: sp := sp + (-1) ; {1111010 = PUSH}
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)	61: mar := sp ; mbr := ac ; wr ; goto 10 ;
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)	62: mar := sp ; sp := sp + 1 ; rd ; {1111011 = POP}
				63: rd ;
				64: ac := mbr ; goto 0 ;
				65: tir := <i>lshift</i> (tir) ; if n then goto 73 ;
				66: alu := tir ; if n then goto 70 ;
				67: mar := sp ; sp := sp + 1 ; rd ; {1111100 = RETN}
				68: rd ;
				69: pc := mbr ; goto 0 ;
				70: a := ac ; {1111101 = SWAP}
				71: ac := sp ;
				72: sp := a ; goto 0 ;
				73: alu := tir ; if n then goto 76 ;
				74: a := <i>band</i> (ir , smask) ; {1111110 = INSP}
				75: sp := sp + a ; goto 0 ;
				76: a := <i>band</i> (ir , smask) ; {1111111 = DESP}
				77: a := <i>inv</i> (a) ;
				78: a := a + 1 ; goto 75 ;

0: mar := pc ; rd ;	{ciclo pc ; principal}
1: pc := pc + 1 ; rd ;	{incrementa pc}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}
6: mar := ir ; rd ;	{0000 = LODD}
7: rd ;	
8: ac := mbr ; goto 0 ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}
10: wr ; goto 0 ;	
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}
12: mar := ir ; rd ;	{0010 = ADDD}
13: rd ;	
14: ac := mbr + ac ; goto 0 ;	
15: mar := ir ; rd ;	{0011 = SUBD}
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;	
18: ac := ac + a ; goto 0 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}
24: goto 0 ;	{no se produce el salto}
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}
30: alu := tir ; if n then goto 33 ;	{1000 o 1001?}
31: a := ir + sp ;	{1000 = LODL}
32: mar := a ; rd ; goto 7 ;	
33: a := ir + sp ;	{1001 = STOL}
34: mar := a ; mbr := ac ; wr ; goto 10 ;	
35: alu := tir ; if n then goto 38 ;	{1010 o 1011?}
36: a := ir + sp ;	{1010 = ADDL}
37: mar := a ; rd ; goto 13 ;	
38: a := ir + sp ;	{1011 = SUBL}
39: mar := a ; rd ; goto 16 ;	

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac>=0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<=x<=4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:=ac+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

```

0: mar := pc ; rd ;                               {ciclo principal}
1: pc := pc + 1 ; rd ;                             {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                 {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;         {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                 {0000 o 0001?}
6: mar := ir ; rd ;                               {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                   {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;               {0010 o 0011?}
12: mar := ir ; rd ;                             {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                             {0011 = SUBD}
16: ac := ac + 1 ; rd ;                           {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;       {010x o 011x?}
20: alu := tir ; if n then goto 23 ;               {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;              {realiza el salto}
23: alu := ac ; if z then goto 22 ;               {0101 = JZER}
24: goto 0 ;                                       {no se produce el salto}
25: alu := tir ; if n then goto 27 ;               {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;              {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;              {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;   {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;       {100x o 101x?}
30: alu := tir ; if n then goto 33 ;               {1000 o 1001?}
31: a := ir + sp ;                               {1000 = LODL}
32: mar := a ; rd ; goto 7 ;
33: a := ir + sp ;                               {1001 = STOL}
34: mar := a ; mbr := ac ; wr ; goto 10 ;
35: alu := tir ; if n then goto 38 ;               {1010 o 1011?}
36: a := ir + sp ;                               {1010 = ADDL}
37: mar := a ; rd ; goto 13 ;
38: a := ir + sp ;                               {1011 = SUBL}
39: mar := a ; rd ; goto 16 ;

```

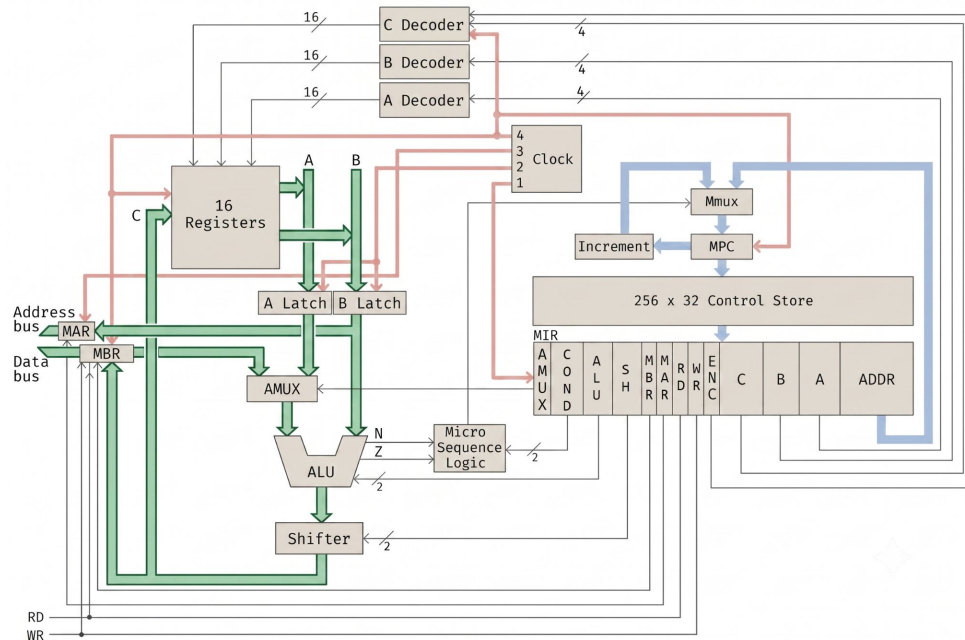
Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx		Add direct	ac:=ac+m[x]
0011xxxx			
0100xxxx			if ac<0 then pc:=x
0101xxxx			if ac<>0 then pc:=x
0110xxxx			
0111xxxx			if ac<=4095)
1000xxxx			
1001xxxx			ac
1010xxxx			pc+ x]
1011xxxx			pc+ x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

3F7 ...
3F8 STOD 9
3F9 JUMP 3F7
3FA ...

0: mar := pc ; rd ;	{ciclo principal}
1: pc := pc + 1 ; rd ;	{incrementa pc}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}
6: mar := ir ; rd ;	{0000 = LODD}
7: rd ;	
8: ac := mbr ; goto 0 ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}
10: wr ; goto 0 ;	
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}
12: mar := ir ; rd ;	{0010 = ADDD}
13: rd ;	
14: ac := mbr + ac ; goto 0 ;	
15: mar := ir ; rd ;	{0011 = SUBD}
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;	
18: ac := ac + a ; goto 0 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}
24: goto 0 ;	{no se produce el salto}
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}
30: alu := tir ; if n then goto 33 ;	{1000 o 1001?}
31: a := ir + sp ;	{1000 = LODL}
32: mar := a ; rd ; goto 7 ;	
33: a := ir + sp ;	{1001 = STOL}
34: mar := a ; mbr := ac ; wr ; goto 10 ;	
35: alu := tir ; if n then goto 38 ;	{1010 o 1011?}
36: a := ir + sp ;	{1010 = ADDL}
37: mar := a ; rd ; goto 13 ;	
38: a := ir + sp ;	{1011 = SUBL}
39: mar := a ; rd ; goto 16 ;	

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx		Add direct	ac:=ac+m[x]
0011xxxx			
0100xxxx			then pc:=x
0101xxxx			en pc:=x
0110xxxx			
0111xxxx			x<=4095)
1000xxxx			
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]

3F7 ...
 3F8 STOD 9
 3F9 JUMP 3F7
 3FA ...

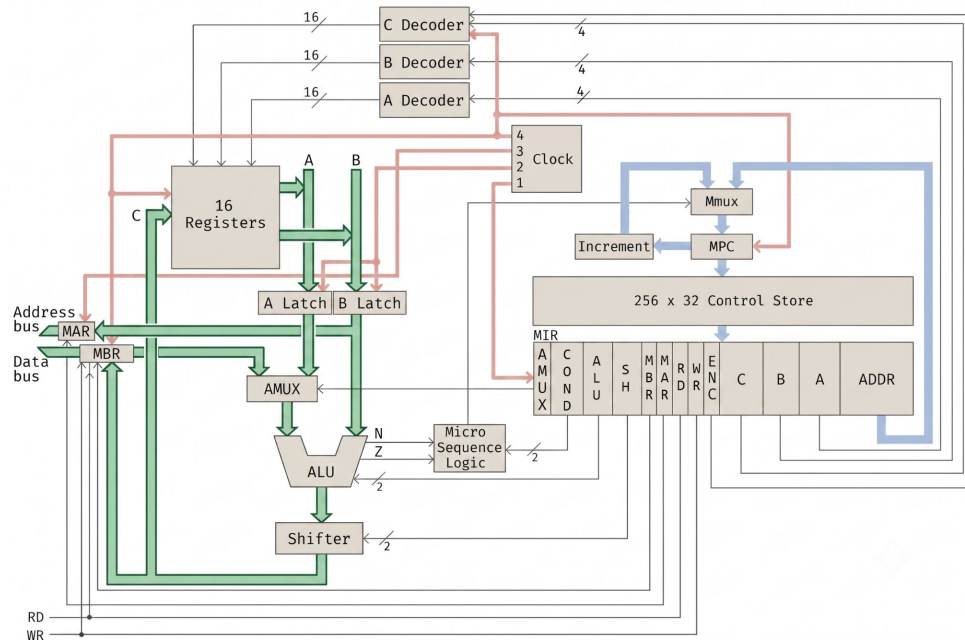


0: mar := pc ; rd ;	{ciclo principal}
1: pc := pc + 1 ; rd ;	{incrementa pc}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}
6: mar := ir ; rd ;	{0000 = LODD}
7: rd ;	
8: ac := mbr ; goto 0 ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}
10: wr ; goto 0 ;	
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}
12: mar := ir ; rd ;	{0010 = ADDD}
13: rd ;	
14: ac := mbr + ac ; goto 0 ;	
15: mar := ir ; rd ;	{0011 = SUBD}
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;	
18: ac := ac + a ; goto 0 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}
24: goto 0 ;	{no se produce el salto}
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}

M	M	P	A	I	T	A	S	N	Z
A	B	C	C	R	R	L	H		
R	R					U	R		
		3F8	0041						

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx		Add direct	ac:=ac+m[x]
0011xxxx			
0100xxxx			then pc:=x
0101xxxx			en pc:=x
0110xxxx			
0111xxxx			x<=4095)
1000xxxx			
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]

3F7 ...
 3F8 STOD 9
 3F9 JUMP 3F7
 3FA ...



0: mar := pc ; rd ;	{ciclo principal}
1: pc := pc + 1 ; rd ;	{incrementa pc}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}
6: mar := ir ; rd ;	{0000 = LODD}
7: rd ;	
8: ac := mbr ; goto 0 ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}
10: wr ; goto 0 ;	
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}
12: mar := ir ; rd ;	{0010 = ADDD}
13: rd ;	
14: ac := mbr + ac ; goto 0 ;	
15: mar := ir ; rd ;	{0011 = SUBD}
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;	
18: ac := ac + a ; goto 0 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}
24: goto 0 ;	{no se produce el salto}
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}

M	M	P	A	I	T	A	S	N	Z
A	B	C	C	R	R	L	H		
R	R					U	R		
		3F8	0041						

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx			
0011xxxx			
0100xxxx			
0101xxxx			
0110xxxx			
0111xxxx			
1000xxxx			
1001xxxx			
1010xxxx			
1011xxxx			

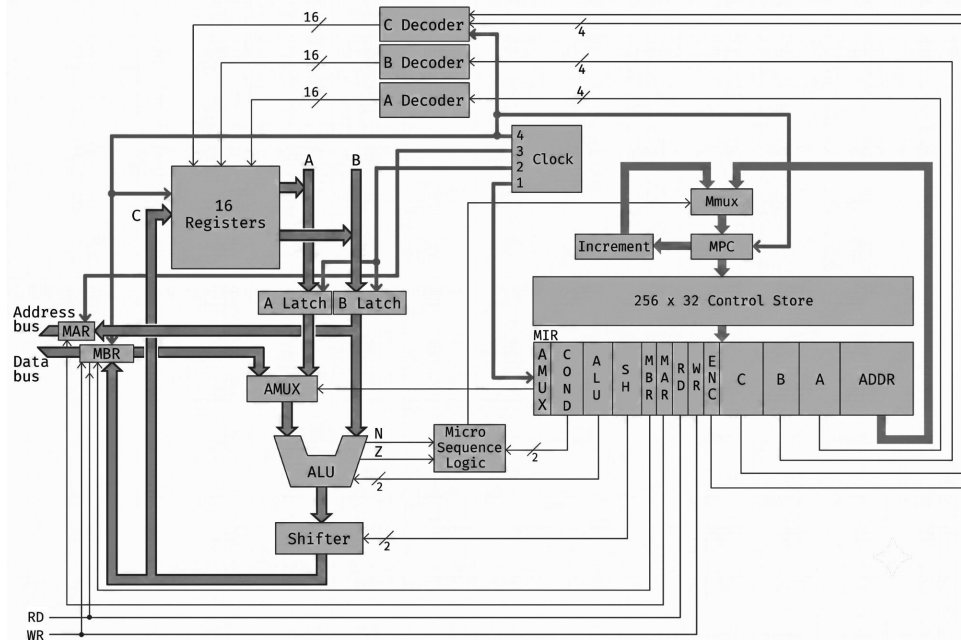
STORE DIRECT

3F8 STOD 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001



0: mar := pc ; rd ;	{ciclo principal}
1: pc := pc + 1 ; rd ;	{incrementa pc}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}
6: mar := ir ; rd ;	{0000 = LODD}
7: rd ;	
8: ac := mbr ; goto 0 ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}
10: wr ; goto 0 ;	
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}
12: mar := ir ; rd ;	{0010 = ADDD}
13: rd ;	
14: ac := mbr + ac ; goto 0 ;	
15: mar := ir ; rd ;	{0011 = SUBD}
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;	
18: ac := ac + a ; goto 0 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}
24: goto 0 ;	{no se produce el salto}
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}

M A R	M B R	P C	A C	I R	T I R	A L U	S H R	N	Z
3F8		3F8	00 41						

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx		Add direct	ac:=ac+m[x]
0011xxxx			
0100xxxx			then pc:=x
0101xxxx			en pc:=x
0110xxxx			
0111xxxx			x<=4095)
1000xxxx			
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]

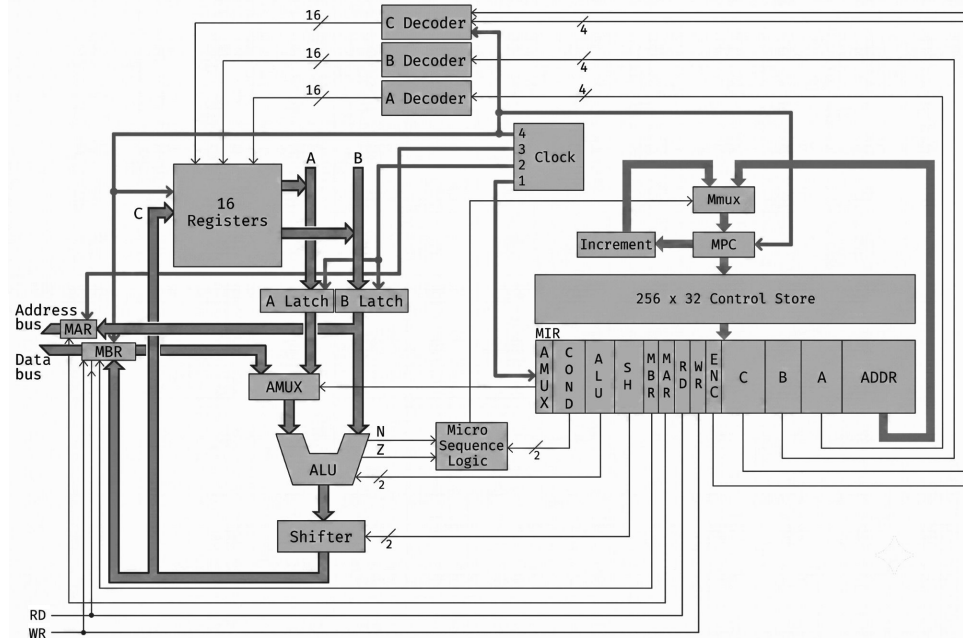
STORE DIRECT

3F8 STOD 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001



0: mar := pc ; rd ;	{ciclo principal}
1: pc := pc + 1 ; rd ;	{incrementa pc}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}
6: mar := ir ; rd ;	{0000 = LODD}
7: rd ;	
8: ac := mbr ; goto 0 ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}
10: wr ; goto 0 ;	
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}
12: mar := ir ; rd ;	{0010 = ADDD}
13: rd ;	
14: ac := mbr + ac ; goto 0 ;	
15: mar := ir ; rd ;	{0011 = SUBD}
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;	
18: ac := ac + a ; goto 0 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}
24: goto 0 ;	{no se produce el salto}
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}

M A R	M B R	P C	A C	I R	T I R	A L U	S H R	N	Z
3F8	10 09	3F9	00 41						

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx		Add direct	ac:=ac+m[x]
0011xxxx			
0100xxxx			then pc:=x
0101xxxx			en pc:=x
0110xxxx			
0111xxxx			x<=4095)
1000xxxx			
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]

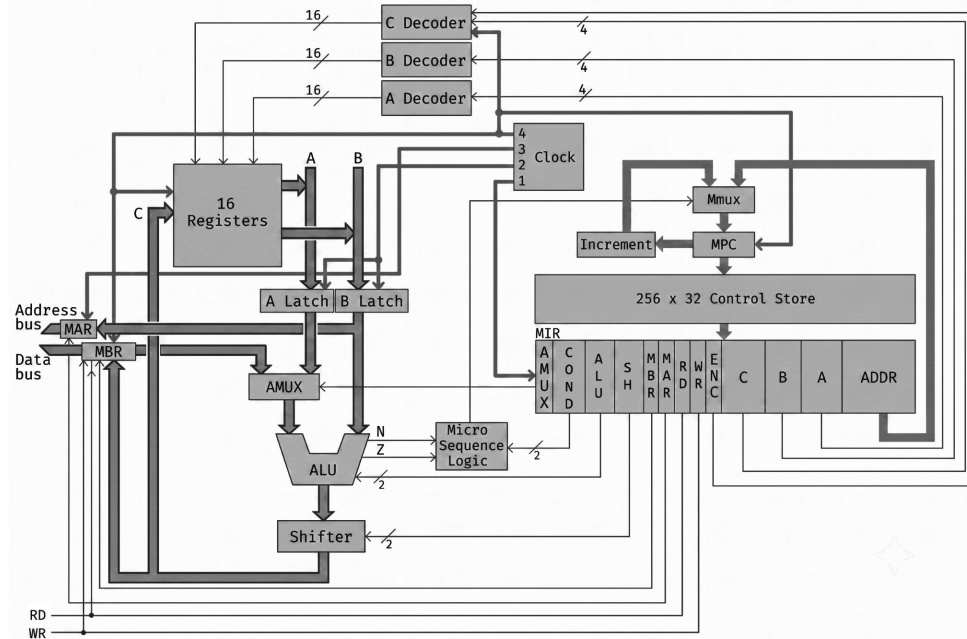
STORE DIRECT

3F8 STOD 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001



0: mar := pc ; rd ;	{ciclo principal}
1: pc := pc + 1 ; rd ;	{incrementa pc}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}
6: mar := ir ; rd ;	{0000 = LODD}
7: rd ;	
8: ac := mbr ; goto 0 ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}
10: wr ; goto 0 ;	
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}
12: mar := ir ; rd ;	{0010 = ADDD}
13: rd ;	
14: ac := mbr + ac ; goto 0 ;	
15: mar := ir ; rd ;	{0011 = SUBD}
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;	
18: ac := ac + a ; goto 0 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}
24: goto 0 ;	{no se produce el salto}
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}

M A R	M B R	P C	A C	I R	T I R	A L U	S H R	N	Z
3F8	10 09	3F9	00 41	10 09		10 09	10 09	0	

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx		Add direct	ac:=ac+m[x]
0011xxxx			
0100xxxx			then pc:=x
0101xxxx			en pc:=x
0110xxxx			
0111xxxx			x<=4095)
1000xxxx			
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]

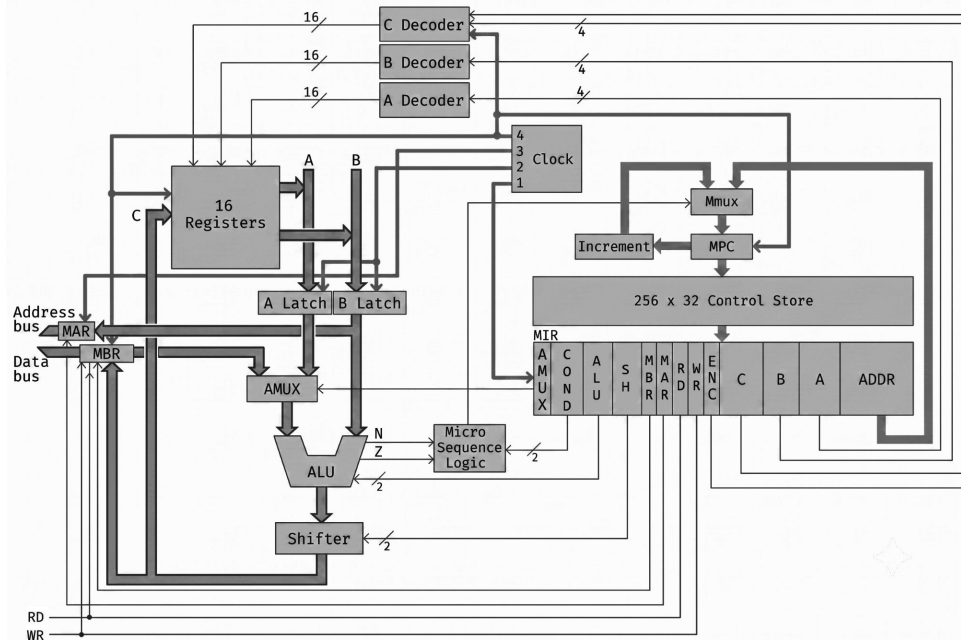
STORE DIRECT

3F8 STOD 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001



0: mar := pc ; rd ;	{ciclo principal}
1: pc := pc + 1 ; rd ;	{incrementa pc}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}
6: mar := ir ; rd ;	{0000 = LODD}
7: rd ;	
8: ac := mbr ; goto 0 ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}
10: wr ; goto 0 ;	
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}
12: mar := ir ; rd ;	{0010 = ADDD}
13: rd ;	
14: ac := mbr + ac ; goto 0 ;	
15: mar := ir ; rd ;	{0011 = SUBD}
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;	
18: ac := ac + a ; goto 0 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}
24: goto 0 ;	{no se produce el salto}
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}

M A R	M B R	P C	A C	I R	T I R	A L U	S H R	N	Z
3F8	10 09	3F9	00 41	10 09	40 24	20 12	40 24	0	

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx		Add direct	ac:=ac+m[x]
0011xxxx			
0100xxxx			then pc:=x
0101xxxx			en pc:=x
0110xxxx			
0111xxxx			x<=4095)
1000xxxx			
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]

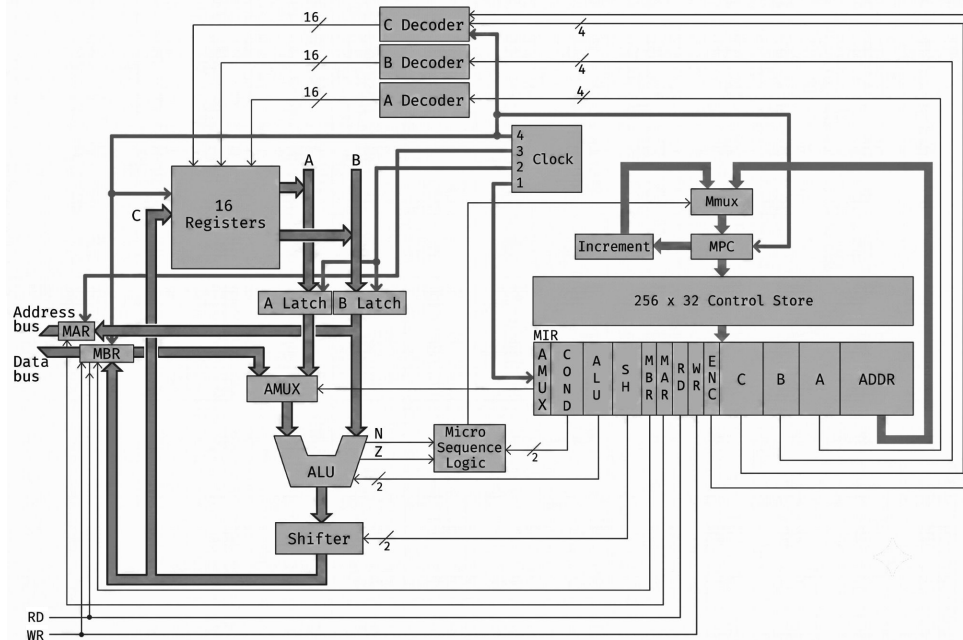
STORE DIRECT

3F8 STOD 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001



0: mar := pc ; rd ;	{ciclo principal}
1: pc := pc + 1 ; rd ;	{incrementa pc}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}
6: mar := ir ; rd ;	{0000 = LODD}
7: rd ;	
8: ac := mbr ; goto 0 ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}
10: wr ; goto 0 ;	
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}
12: mar := ir ; rd ;	{0010 = ADDD}
13: rd ;	
14: ac := mbr + ac ; goto 0 ;	
15: mar := ir ; rd ;	{0011 = SUBD}
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;	
18: ac := ac + a ; goto 0 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}
24: goto 0 ;	{no se produce el salto}
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}

M A R	M B R	P C	A C	I R	T I R	A L U	S H R	N	Z
3F8	10 09	3F9	00 41	10 09	80 48	40 24	80 48	0	

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx		Load indirect	ac:=m[m[x]]
0011xxxx		Store indirect	m[m[x]]:=ac
0100xxxx		Jump	when pc:=x
0101xxxx		Zero	when pc:=x
0110xxxx		Positive	when pc:=x
0111xxxx		Loco	when pc:=x
1000xxxx		Shift right	ac<=4095)
1001xxxx		Shift left	
1010xxxx		Shift right	p+x]
1011xxxx		Shift left	p+x]

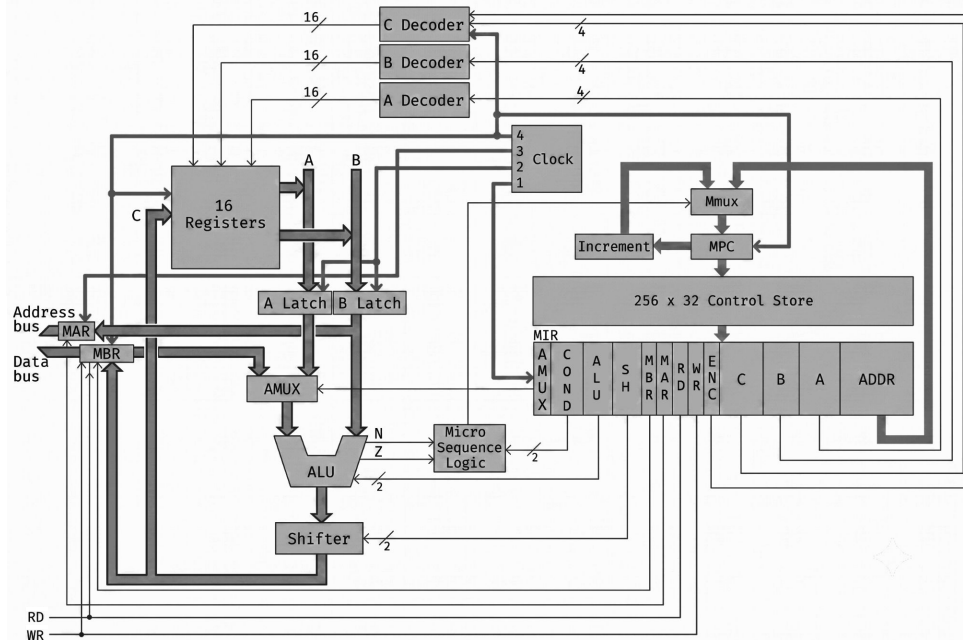
STORE DIRECT

3F8 STOD 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001



0: mar := pc ; rd ;	{ciclo principal}
1: pc := pc + 1 ; rd ;	{incrementa pc}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}
6: mar := ir ; rd ;	{0000 = LODD}
7: rd ;	
8: ac := mbr ; goto 0 ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}
10: wr ; goto 0 ;	
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}
12: mar := ir ; rd ;	{0010 = ADDD}
13: rd ;	
14: ac := mbr + ac ; goto 0 ;	
15: mar := ir ; rd ;	{0011 = SUBD}
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;	
18: ac := ac + a ; goto 0 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}
24: goto 0 ;	{no se produce el salto}
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}

M A R	M B R	P C	A C	I R	T I R	A L U	S H R	N	Z
3F8	10 09	3F9	00 41	10 09	80 48	80 48	80 48	1	

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx		Add direct	ac:=ac+m[x]
0011xxxx			
0100xxxx			then pc:=x
0101xxxx			en pc:=x
0110xxxx			
0111xxxx			x<=4095)
1000xxxx			
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]

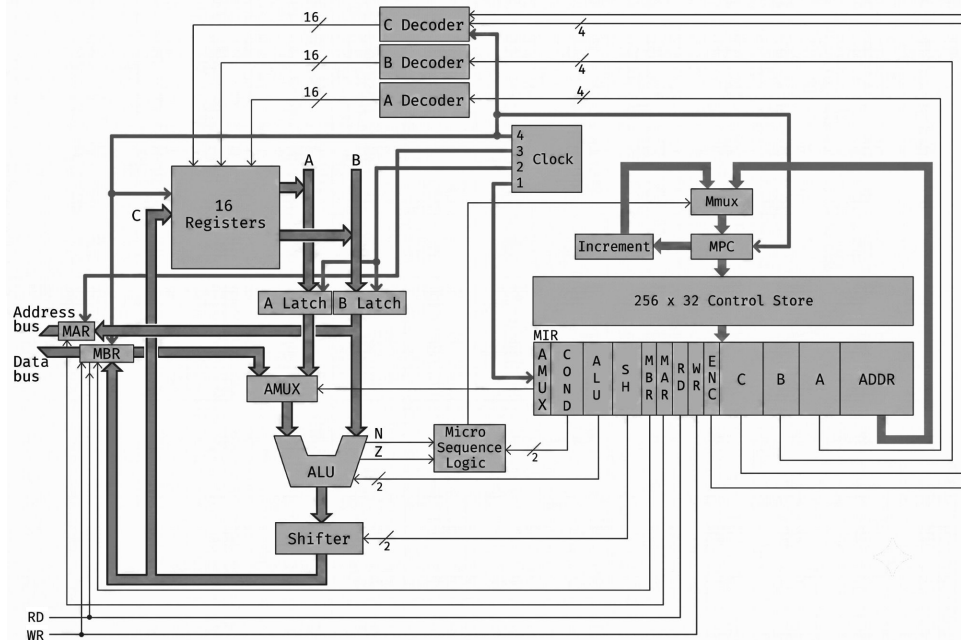
STORE DIRECT

3F8 STOD 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001



0: mar := pc ; rd ;	{ciclo principal}
1: pc := pc + 1 ; rd ;	{incrementa pc}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}
6: mar := ir ; rd ;	{0000 = LODD}
7: rd ;	
8: ac := mbr ; goto 0 ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}
10: wr ; goto 0 ;	
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}
12: mar := ir ; rd ;	{0010 = ADDD}
13: rd ;	
14: ac := mbr + ac ; goto 0 ;	
15: mar := ir ; rd ;	{0011 = SUBD}
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;	
18: ac := ac + a ; goto 0 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}
24: goto 0 ;	{no se produce el salto}
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}

M A R	M B R	P C	A C	I R	T I R	A L U	S H R	N	Z
009	00 41	3F9	00 41	10 09	80 48	00 41	00 41	0	

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx		Add direct	ac:=ac+m[x]
0011xxxx			
0100xxxx			then pc:=x
0101xxxx			en pc:=x
0110xxxx			x<=4095)
0111xxxx			
1000xxxx			
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]

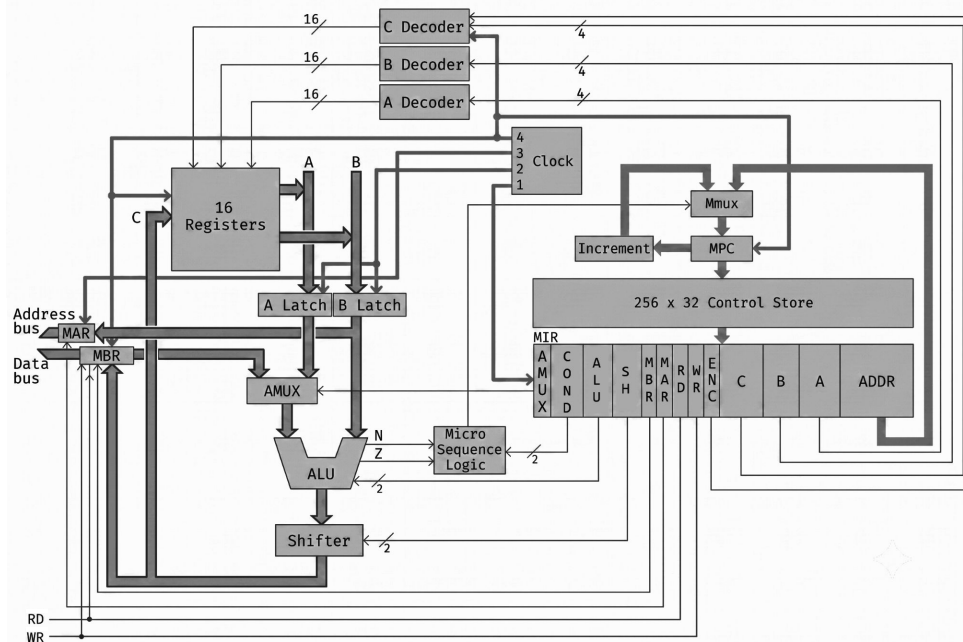
STORE DIRECT

3F8 STOD 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001



0: mar := pc ; rd ;	{ciclo principal}
1: pc := pc + 1 ; rd ;	{incrementa pc}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}
6: mar := ir ; rd ;	{0000 = LODD}
7: rd ;	
8: ac := mbr ; goto 0 ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}
10: wr ; goto 0 ;	
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}
12: mar := ir ; rd ;	{0010 = ADDD}
13: rd ;	
14: ac := mbr + ac ; goto 0 ;	
15: mar := ir ; rd ;	{0011 = SUBD}
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;	
18: ac := ac + a ; goto 0 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}
24: goto 0 ;	{no se produce el salto}
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}

M A R	M B R	P C	A C	I R	T I R	A L U	S H R	N	Z
009	00 41	3F9	00 41	10 09	80 48	00 41	00 41	0	

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx		Add direct	ac:=ac+m[x]
0011xxxx			
0100xxxx			then pc:=x
0101xxxx			en pc:=x
0110xxxx			x<=4095)
0111xxxx			
1000xxxx			
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]

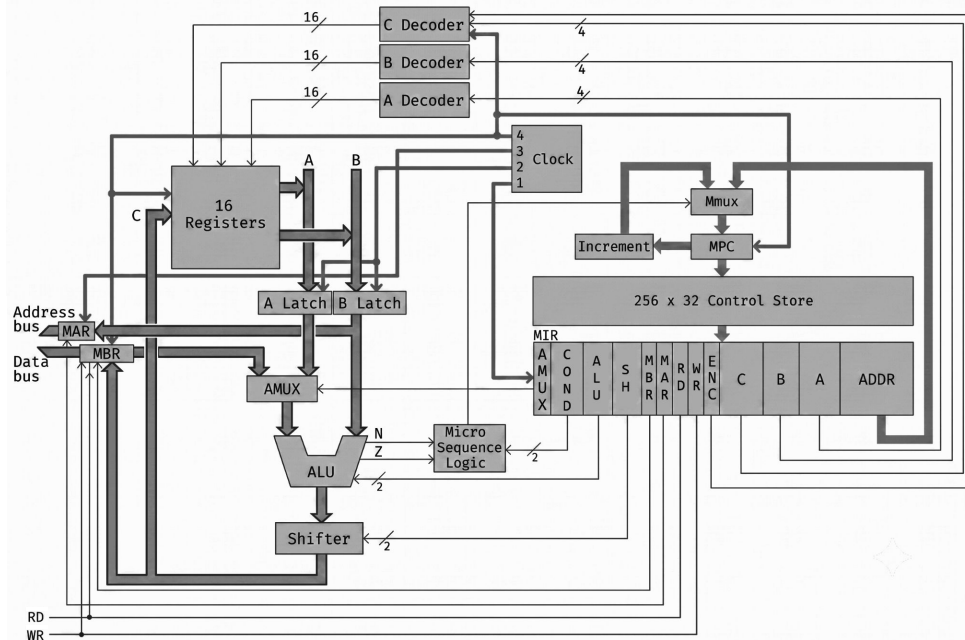
STORE DIRECT

3F8 STOD 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001



0: mar := pc ; rd ;	{ciclo principal}
1: pc := pc + 1 ; rd ;	{incrementa pc}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}
6: mar := ir ; rd ;	{0000 = LODD}
7: rd ;	
8: ac := mbr ; goto 0 ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}
10: wr ; goto 0 ;	
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}
12: mar := ir ; rd ;	{0010 = ADDD}
13: rd ;	
14: ac := mbr + ac ; goto 0 ;	
15: mar := ir ; rd ;	{0011 = SUBD}
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;	
18: ac := ac + a ; goto 0 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}
24: goto 0 ;	{no se produce el salto}
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}

M	M	P	A	I	T	A	S	N	Z
A	B	C	C	R	R	L	H		
R	R					U	R		
3F9	0041	3F9	0041	1009	8048	0041	0041	0	

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx		Add direct	ac:=ac+m[x]
0011xxxx			
0100xxxx			then pc:=x
0101xxxx			en pc:=x
0110xxxx			
0111xxxx			x<=4095)
1000xxxx			
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]

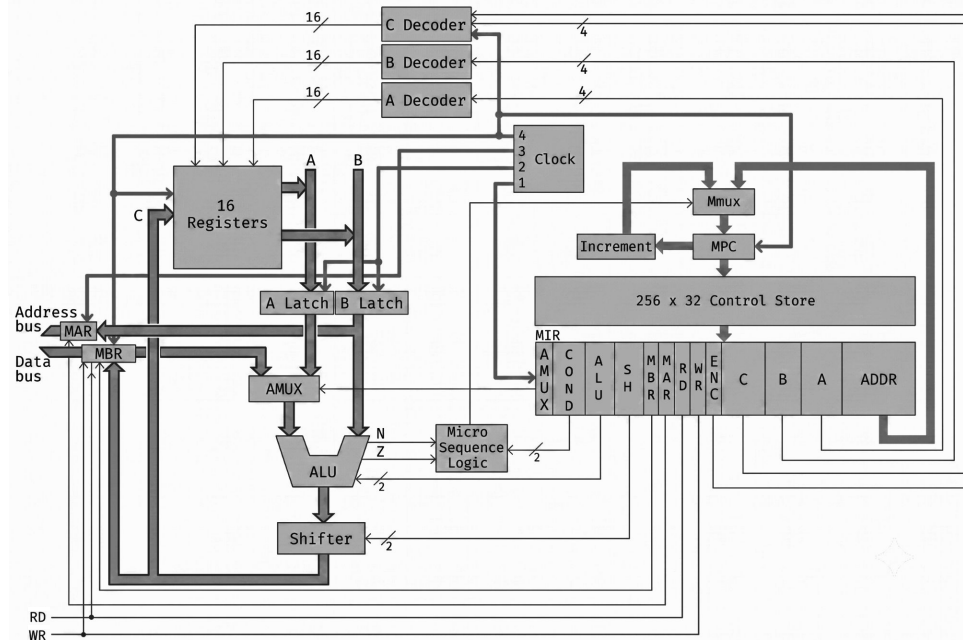
STORE DIRECT

3F8 STOD 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001



0: mar := pc ; rd ;	{ciclo principal}
1: pc := pc + 1 ; rd ;	{incrementa pc}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}
6: mar := ir ; rd ;	{0000 = LODD}
7: rd ;	
8: ac := mbr ; goto 0 ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}
10: wr ; goto 0 ;	
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}
12: mar := ir ; rd ;	{0010 = ADDD}
13: rd ;	
14: ac := mbr + ac ; goto 0 ;	
15: mar := ir ; rd ;	{0011 = SUBD}
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;	
18: ac := ac + a ; goto 0 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}
24: goto 0 ;	{no se produce el salto}
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}

M A R	M B R	P C	A C	I R	T I R	A L U	S H R	N	Z
3F9	63 F7	3FA	00 41	10 09	80 48	00 41	00 41	0	

Binario	Mnem	Instrucción	Significado
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxx			
1000xxxx			
1001xxxx			
1010xxxx			
1011xxxx			
1100xxxx			
1101xxxx			
1110xxxx			
11110000			
11110010			

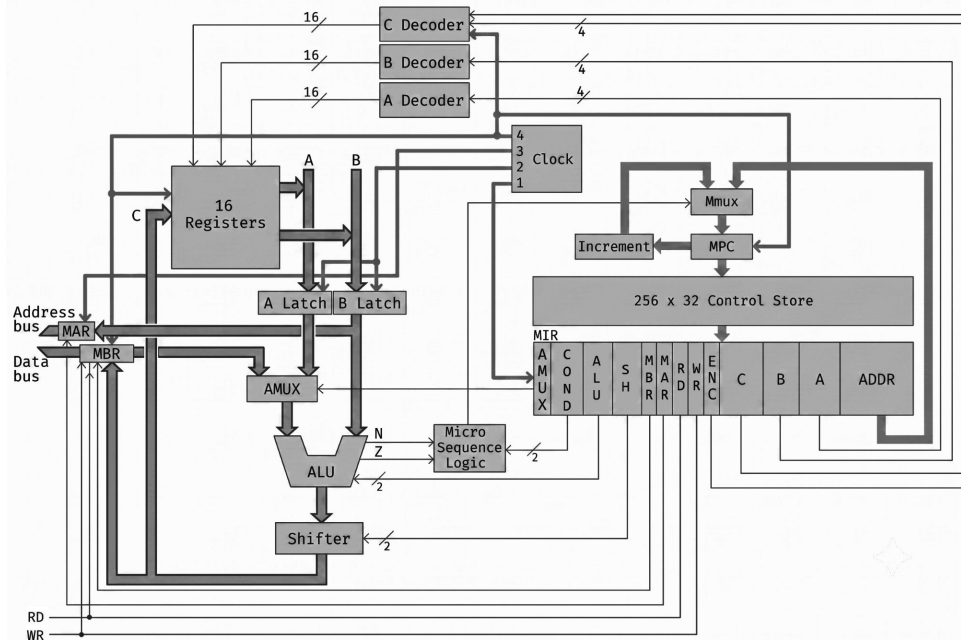
JUMP

3F9 JUMP 3F7

Salto incondicional

- pc:=band(ir , amask)

0110 0011 1111 0111



0: mar := pc ; rd ;	{ciclo principal}
1: pc := pc + 1 ; rd ;	{incrementa pc}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}
6: mar := ir ; rd ;	{0000 = LODD}
7: rd ;	
8: ac := mbr ; goto 0 ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}
10: wr ; goto 0 ;	
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}
12: mar := ir ; rd ;	{0010 = ADDD}
13: rd ;	
14: ac := mbr + ac ; goto 0 ;	
15: mar := ir ; rd ;	{0011 = SUBD}
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;	
18: ac := ac + a ; goto 0 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}
24: goto 0 ;	{no se produce el salto}
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}

M A R	M B R	P C	A C	I R	T I R	A L U	S H R	N	Z
3F9	63 F7	3FA	00 41	63 F7	80 48	63 F7	63 F7	0	

Binario	Mnem	Instrucción	Significado
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxx			
1000xxxx			
1001xxxx			
1010xxxx			
1011xxxx			
1100xxxx			
1101xxxx			
1110xxxx			
11110000			
11110010			

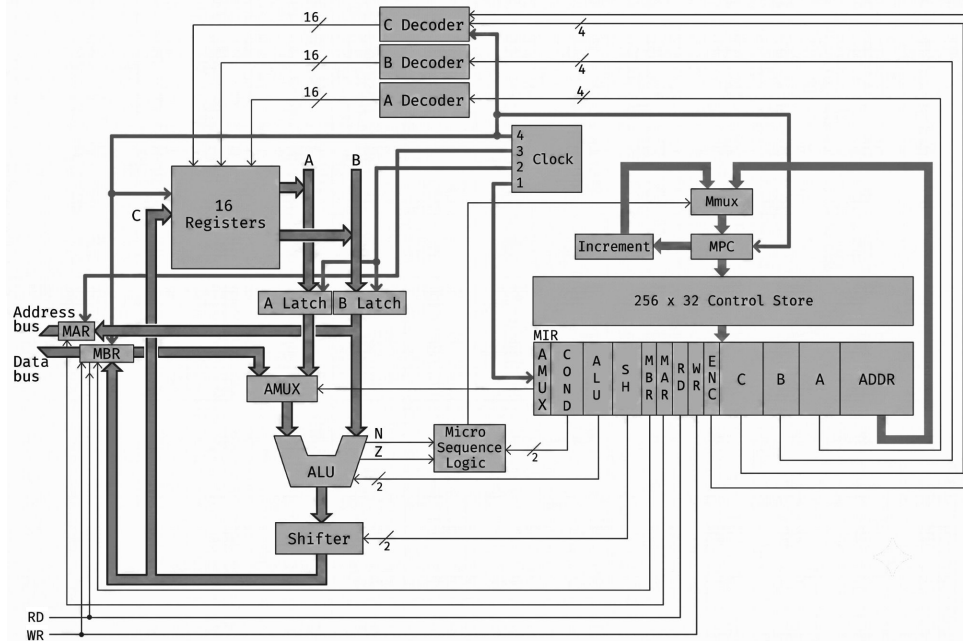
JUMP

3F9 JUMP 3F7

Salto incondicional

- pc:=band(ir , amask)

0110 0011 1111 0111



0: mar := pc ; rd ;	{ciclo principal}
1: pc := pc + 1 ; rd ;	{incrementa pc}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}
6: mar := ir ; rd ;	{0000 = LODD}
7: rd ;	
8: ac := mbr ; goto 0 ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}
10: wr ; goto 0 ;	
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}
12: mar := ir ; rd ;	{0010 = ADDD}
13: rd ;	
14: ac := mbr + ac ; goto 0 ;	
15: mar := ir ; rd ;	{0011 = SUBD}
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;	
18: ac := ac + a ; goto 0 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}
24: goto 0 ;	{no se produce el salto}
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}

M A R	M B R	P C	A C	I R	T I R	A L U	S H R	N	Z
3F9	63 F7	3FA	00 41	63 F7	8F DC	C7 EE	8F DC	1	

Binario	Mnem	Instrucción	Significado
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxx			
1000xxxx			
1001xxxx			
1010xxxx			
1011xxxx			
1100xxxx			
1101xxxx			
1110xxxx			
11110000			
11110010			

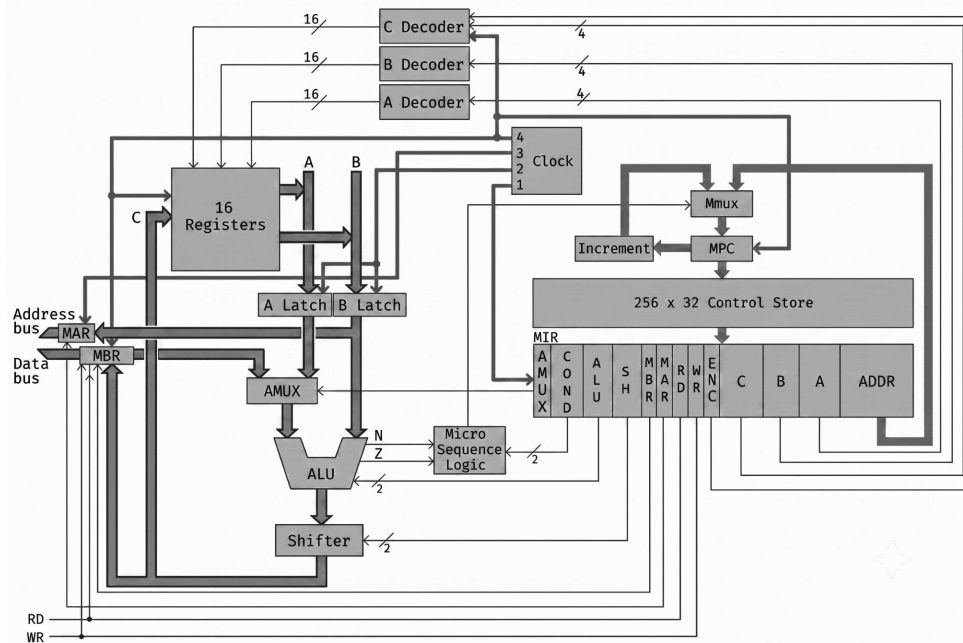
JUMP

3F9 JUMP 3F7

Salto incondicional

- pc:=band(ir , amask)

0110 0011 1111 0111



0: mar := pc ; rd ;	{ciclo principal}
1: pc := pc + 1 ; rd ;	{incrementa pc}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}
6: mar := ir ; rd ;	{0000 = LODD}
7: rd ;	
8: ac := mbr ; goto 0 ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}
10: wr ; goto 0 ;	
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}
12: mar := ir ; rd ;	{0010 = ADDD}
13: rd ;	
14: ac := mbr + ac ; goto 0 ;	
15: mar := ir ; rd ;	{0011 = SUBD}
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;	
18: ac := ac + a ; goto 0 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}
24: goto 0 ;	{no se produce el salto}
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}

M A R	M B R	P C	A C	I R	T I R	A L U	S H R	N	Z
3F9	63 F7	3FA	00 41	63 F7	1F B8	8F DC	1F B8	1	

Binario	Mnem	Instrucción	Significado
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxx			
1000xxxx			
1001xxxx			
1010xxxx			
1011xxxx			
1100xxxx			
1101xxxx			
1110xxxx			
11110000			
11110010			

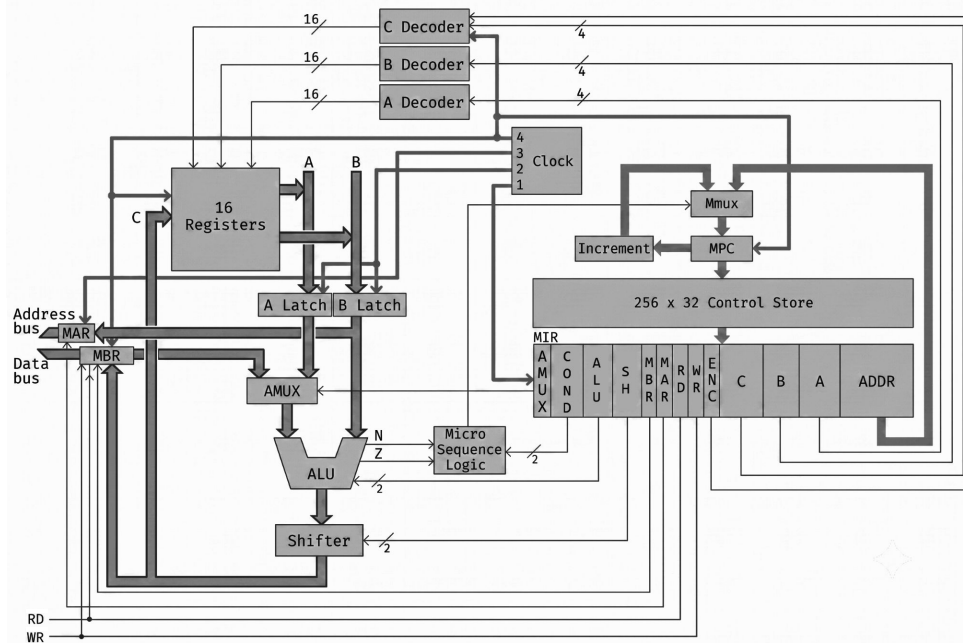
JUMP

3F9 JUMP 3F7

Salto incondicional

- pc:=band(ir , amask)

0110 0011 1111 0111



0: mar := pc ; rd ;	{ciclo principal}
1: pc := pc + 1 ; rd ;	{incrementa pc}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}
6: mar := ir ; rd ;	{0000 = LODD}
7: rd ;	
8: ac := mbr ; goto 0 ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}
10: wr ; goto 0 ;	
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}
12: mar := ir ; rd ;	{0010 = ADDD}
13: rd ;	
14: ac := mbr + ac ; goto 0 ;	
15: mar := ir ; rd ;	{0011 = SUBD}
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;	
18: ac := ac + a ; goto 0 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}
24: goto 0 ;	{no se produce el salto}
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}

M A R	M B R	P C	A C	I R	T I R	A L U	S H R	N	Z
3F9	63 F7	3FA	00 41	63 F7	1F B8	1F B8		0	

Binario	Mnem	Instrucción	Significado
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxx			
1000xxxx			
1001xxxx			
1010xxxx			
1011xxxx			
1100xxxx			
1101xxxx			
1110xxxx			
11110000			
11110010			

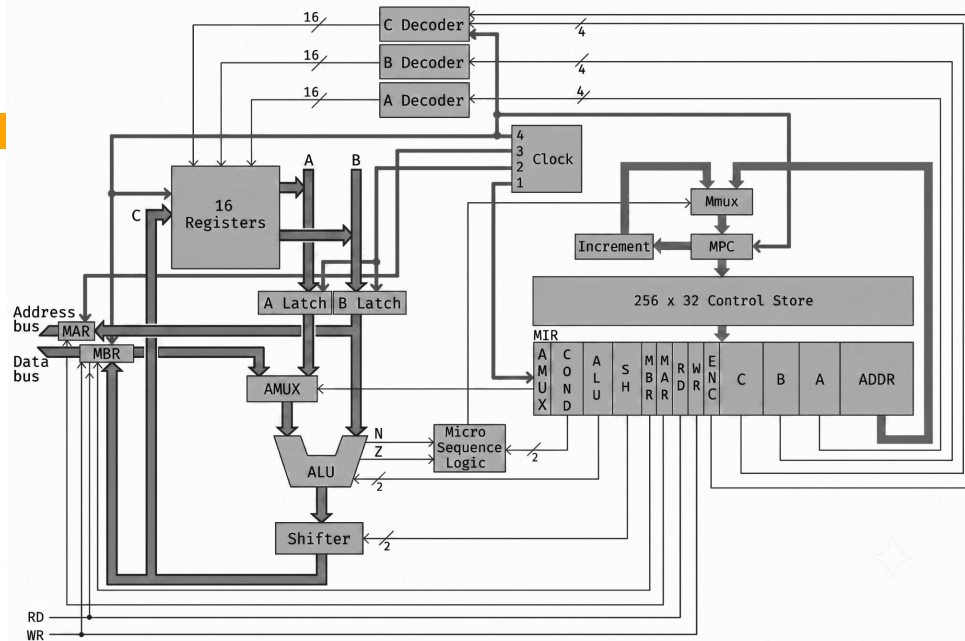
JUMP

3F9 JUMP 3F7

Salto incondicional

- pc:=band(ir , amask)

0110 0011 1111 0111



0: mar := pc ; rd ;	{ciclo principal}
1: pc := pc + 1 ; rd ;	{incrementa pc}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}
6: mar := ir ; rd ;	{0000 = LODD}
7: rd ;	
8: ac := mbr ; goto 0 ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}
10: wr ; goto 0 ;	
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}
12: mar := ir ; rd ;	{0010 = ADDD}
13: rd ;	
14: ac := mbr + ac ; goto 0 ;	
15: mar := ir ; rd ;	{0011 = SUBD}
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;	
18: ac := ac + a ; goto 0 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}
24: goto 0 ;	{no se produce el salto}
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}

M A R	M B R	P C	A C	I R	T I R	A L U	S H R	N	Z
3F9	63 F7	3F7	00 41	63 F7	1F B8	03 F7	03 F7	0	

Binario	Mnem	Instrucción	Significado
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxx			
1000xxxxx			
1001xxxxx			
1010xxxxx			
1011xxxxx			
1100xxxxx			
1101xxxxx			
1110xxxxx			
11110000			
11110010			

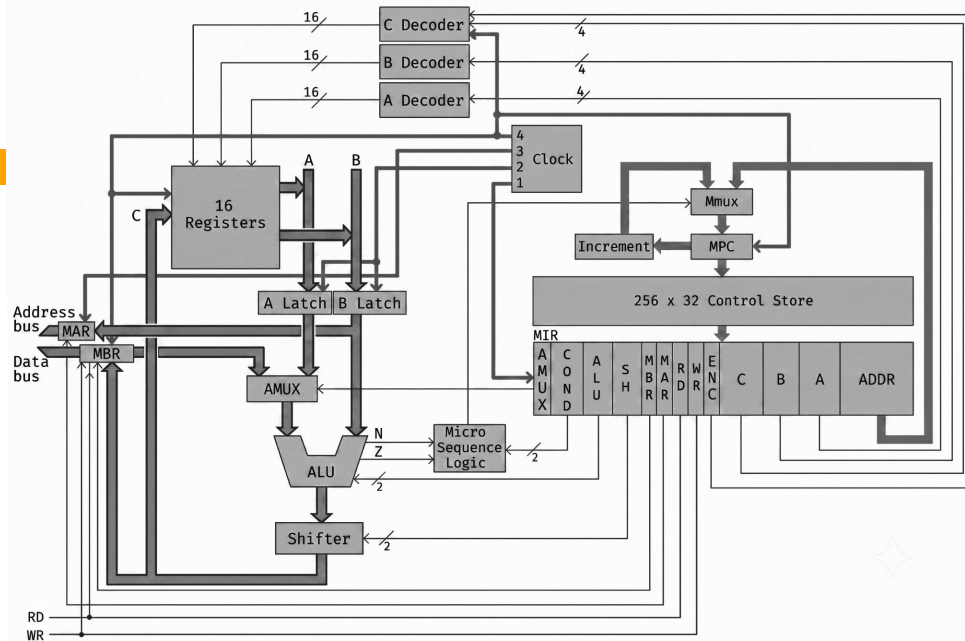
JUMP

3F9 JUMP 3F7

Salto incondicional

- pc:=band(ir , amask)

0110 0011 1111 0111



0: mar := pc ; rd ;	{ciclo principal}
1: pc := pc + 1 ; rd ;	{incrementa pc}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}
6: mar := ir ; rd ;	{0000 = LODD}
7: rd ;	
8: ac := mbr ; goto 0 ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}
10: wr ; goto 0 ;	
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}
12: mar := ir ; rd ;	{0010 = ADDD}
13: rd ;	
14: ac := mbr + ac ; goto 0 ;	
15: mar := ir ; rd ;	{0011 = SUBD}
16: ac := ac + 1 ; rd ;	{Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;	
18: ac := ac + a ; goto 0 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}
24: goto 0 ;	{no se produce el salto}
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}

M A R	M B R	P C	A C	I R	T I R	A L U	S H R	N	Z
3F7	63 F7	3F7	00 41	63 F7	1F B8	03 F7	03 F7	0	

Binario	Mnem	Instrucción	Significado
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxx			
1000xxxxx			
1001xxxxx			
1010xxxxx			
1011xxxxx			
1100xxxxx			
1101xxxxx			
1110xxxxx			
11110000			
11110010			

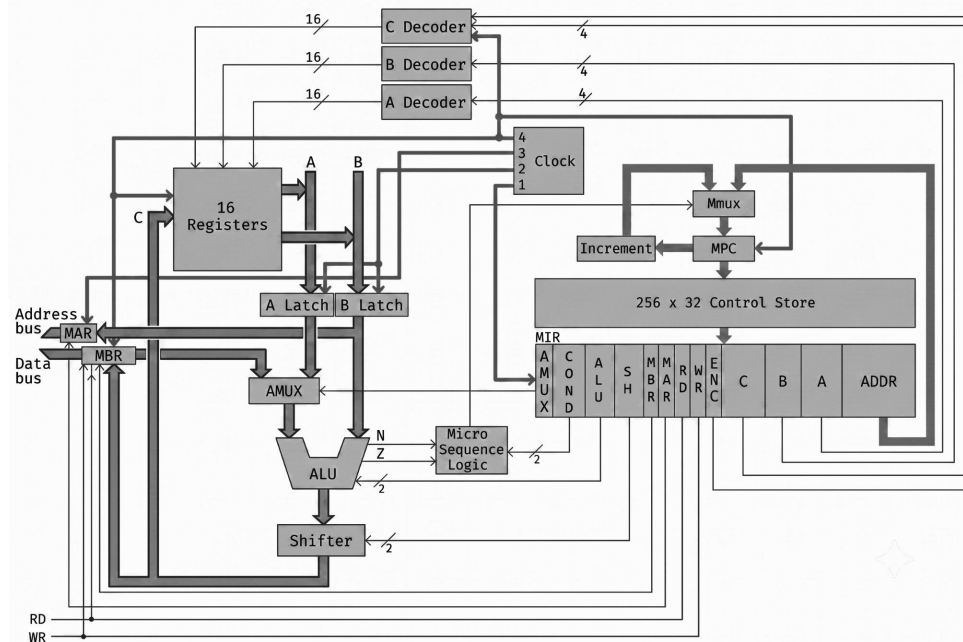
JUMP

3F9 JUMP 3F7

Salto incondicional

- pc:=band(ir , amask)

0110 0011 1111 0111



0: mar := pc ; rd ;	{ciclo principal}
1: pc := pc + 1 ; rd ;	{incrementa pc}
2: ir := mbr ; if n then goto 28 ;	{salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;	
4: tir := lshift(tir) ; if n then goto 11 ;	{000x o 001x?}
5: alu := tir ; if n then goto 9 ;	{0000 o 0001?}
6: mar := ir ; rd ;	{0000 = LODD}
7: rd ;	
8: ac := mbr ; goto 0 ;	
9: mar := ir ; mbr := ac ; wr ;	{0001 = STOD}
10: wr ; goto 0 ;	
11: alu := tir ; if n then goto 15 ;	{0010 o 0011?}
12: mar := ir ; rd ;	{0010 = ADDD}
13: rd ;	
14: ac := mbr + ac ; goto 0 ;	
15: mar := ir ; rd ;	{0011 = SUBD}
16: ac := ac + 1 ; rd ;	{Nota: x = y = x - 1 + no y}
17: a := inv(mbr) ;	
18: ac := ac + a ; goto 0 ;	
19: tir := lshift(tir) ; if n then goto 25 ;	{010x o 011x?}
20: alu := tir ; if n then goto 23 ;	{0100 o 0101?}
21: alu := ac ; if n then goto 0 ;	{0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;	{realiza el salto}
23: alu := ac ; if z then goto 22 ;	{0101 = JZER}
24: goto 0 ;	{no se produce el salto}
25: alu := tir ; if n then goto 27 ;	{0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;	{0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;	{0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;	{10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;	{100x o 101x?}

El ciclo se reinicia

M A R	M B R	P C	A C	I R	T I R	A L U	S H R	N	Z
3F7	63 F7	3F7	00 41	63 F7	1F B8	03 F7	03 F7	0	

Binario	Mnem	Instrucción	Significado
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxx			
1000xxxx			
1001xxxx			
1010xxxx			
1011xxxx			
1100xxxx			
1101xxxx			
1110xxxx			
11110000			
11110010			

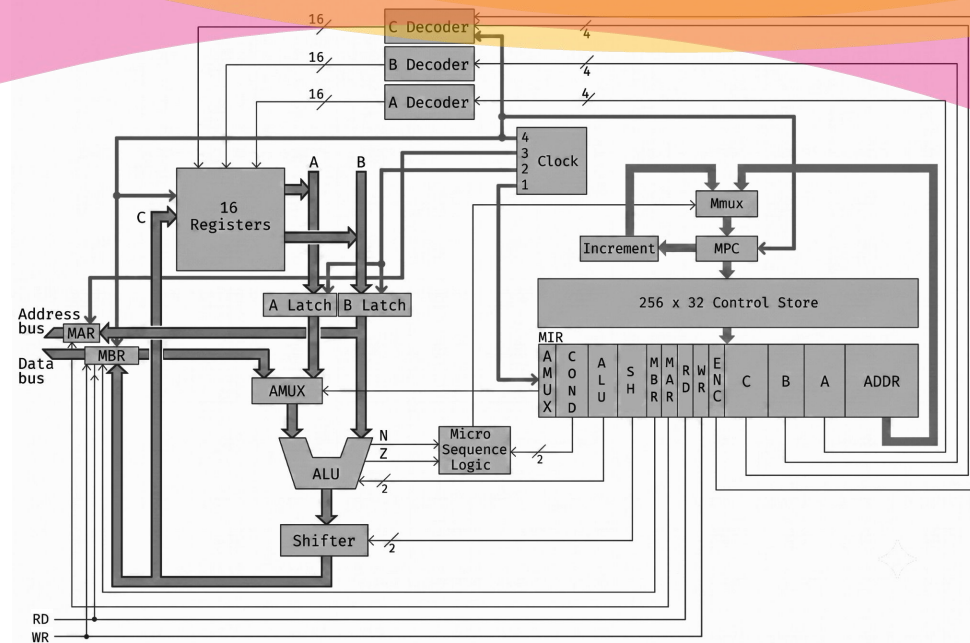
JUMP

3F9 JUMP 3F7

Salto incondicional

- pc:=band(ir , amask)

0110 0011 1111 0111



Arquitectura Horizontal

Ejecución del microprograma



Fundamentos de Arquitectura de computadoras

El control del procesador. Análisis de la secuencias de pasos del micro programa para la ejecución de un programa de usuario.

- Control Store
- Micro instrucción
- Microprograma

